Computers & Security 146 (2024) 104057



Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose





Development and validation of coreLang: A threat modeling language for the ICT domain

Sotirios Katsikeas [a,*], Andrei Buhaiu [a], Mathias Ekstedt [a], Zeeshan Afzal [a,b], Simon Hacks [c], Preetam Mukherjee [d]

[a] KTH Royal Institute of Technology, Division of Network and Systems Engineering, Teknikringen 33, floor 3, SE 100 44 Stockholm, Sweden
[b] Department of Information Technology, Uppsala University, Uppsala 752 36, Sweden
[c] Stockholm University, Stockholm, Sweden
[d] Digital University Kerala, Thiruvananthapuram, India

ARTICLE INFO

Keywords:
Domain specific language
Attack graphs
Cyber attack modeling
Threat modeling
ICT domain

ABSTRACT

ICT infrastructures are getting increasingly complex, and defending them against cyber attacks is cumbersome. As cyber threats continue to increase and expert resources are limited, organizations must find more efficient ways to evaluate their resilience and take proactive measures. Threat modeling is an excellent method of assessing the resilience of ICT systems, for example, by building Attack Graphs that illustrate an adversary's attack vectors. Previously, the Meta Attack Language (MAL) was proposed, which serves as a framework to develop Domain Specific Languages (DSLs) and generate Attack Graphs for modeled infrastructures. coreLang is a MAL-based threat modeling language that utilizes Attack Graphs to enable attack simulations and security assessments. In this work, we present the first release version of coreLang in which MITRE ATT&CK tactics and techniques are mapped onto to serve as a validation and identify strengths and weaknesses to benefit the development cycle. Our validation showed that coreLang does cover 46% of all the techniques included in the matrix, while if we additionally exclude the tactics that are intrinsically not covered by coreLang and MAL, the coverage percentage increases to 64%.

1. Introduction

Protection of Information and Communications Technology (ICT) infrastructures is one of the most critical challenges faced by our society today. Cybersecurity assessments can play a crucial role in securing them. The idea is simple: it will be easier to protect once we understand the limits of our defenses, the gaps (vulnerabilities), and what to protect against. However, as systems are large and complex, this assessment is not a trivial task to undertake (Aksu et al., 2017). Formal methods and automated analysis tools can help experts tackle this complex problem. Indeed, for the task of assessing large ICT infrastructures, threat modeling has emerged as a promising approach, and attack trees and graphs are key formalisms within this field. Popularized by Schneier (1999) over twenty years ago, attack trees are basic encoding of what attackers need to do as steps, combined in logic gates (which, among others, include AND and OR gates), to compromise a certain target. This logic gives rise to several alternative attack vectors. These resultant attack graphs can be very useful for cybersecurity specialists to understand the systems under investigation from the attacker's point of view.

In previous work, we have developed the Meta Attack Language (Johnson et al., 2018) (MAL) and coreLang (Katsikeas et al., 2020). MAL eases the cumbersome task of manually generating a new attack graph for each system. Instead, it provides a framework to define attack languages for different domains (e.g., industrial control systems (ICSs), IoT, etc.) so that attack graphs, that make use of AND and OR logic gates, can be automatically generated from system architecture instance models. Furthermore, as many domains share a lot of common concepts and properties, it makes sense to capture these commonalities at a higher abstraction level rather than individually for all domains before focusing on modeling domain specifics. As the name implies, coreLang is an attack language designed to do exactly that. It covers the basics and common structure of general-purpose software systems and ICT infrastructures and, as such, comprises predefined assets and attacks that appear in most domains.

coreLang, in its current version, is a product of active development over several years and this paper is based on the first stable release of coreLang, version 1.0. This work employs Design Science Research (DSR (Hevner et al., 2004)) and continues our previous work (Katsikeas

* Corresponding author.
E-mail address: sotkat@kth.se (S. Katsikeas).

https://doi.org/10.1016/j.cose.2024.104057
Received 10 February 2024; Received in revised form 11 July 2024; Accepted 12 August 2024
Available online 14 August 2024
0167-4048/© 2024 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

et al., 2020) with a focus on the design cycle of coreLang, iterating between core language development (i.e., design phase (Peffers et al., 2007)) and the validation of the work (i.e., evaluation phase (Peffers et al., 2007)). This involves meetings with experts to gain input and feedback based on practical customer experiences, as detailed later. Moreover, in this work, we introduce coreLang as a fundamental all-purpose threat modeling language (Katsikeas et al., 2020) with the goal of representing real-world attack scenarios. Therefore, our primary objective in this work was to assess the extent to which coreLang, our proposed threat modeling language, can represent the techniques documented in the MITRE ATT&CK framework (MITRE, 2024). This framework has emerged as the most comprehensive collection of real-world attack techniques and is widely regarded as a de facto standard in the cybersecurity community for adversary knowledge (Tayouri et al., 2023; Derbyshire et al., 2021).

By mirroring coreLang against the documented attacks in ATT&CK, we aim to identify the strengths and weaknesses of coreLang. The identification of the weaknesses supports improving specific aspects of the language and also reveals other shortcomings that cannot be covered by coreLang. Thus, it makes clear what coreLang is able to model and analyze and not.

The rest of the paper is structured as follows. Section 2 will provide the necessary background and discuss the related work. This is followed up by Section 3 with a description of the employed methodology and assumptions made. Next, Section 4 provides details of the validation work undertaken as part of coreLang development. Then, Section 5 provides a discussion of our work. Finally, Section 6 provides concluding remarks.

2. Background and related work

This section provides the necessary background by introducing MAL, coreLang overview and related work, to help understand the work presented in this paper.

2.1. Attack graphs & the meta attack language

Attack trees and graphs are key formalisms in the field of threat modeling. As already mentioned, they were first elaborated in Phillips and Swiler (1998) over two decades ago and popularized by Schneier (1999). Later on, attack trees were formalized by Mauw and Oostdijk (2005). An important extension of attack trees was introduced by Kordy et al. (2010) to also include defenses to analyze the effectiveness of various countermeasure strategies. Overall, a large body of work in the area up until 2014 is neatly summarized in Kordy et al. (2014). A fairly large sub-group in the field works with probabilistic attack graphs, oftentimes using Bayesian networks as the underlying formalism for analysis (Dantu et al., 2004; Doynikova and Kotenko, 2017; Lin and Chen, 2018; Liu and Man, 2005; Poolsappasit et al., 2012; Yimin et al., 2019; Singhal and Ou, 2017; Xie et al., 2010).

Attack trees and graphs encode what attackers need to do as steps, combined in logic AND and OR gates, to compromise a certain target. This means that the successful compromise of one attack step can enable attempting another attack step. An encoding that requires the compromise of at least one of its parent attack steps to be required to compromise this step is expressed by an OR attack step, or in the case where the compromise of all parent steps is required, an AND attack step should be used.

This logic gives rise to several alternative attack vectors. To estimate their difficulty, the nodes are associated with resilience metrics such as attacker cost, capability, or time needed to succeed with each step. This metric depends on the attacker's properties and the attack's technical implementation (a highly capable attacker will be more successful in compromising the system, but a better-protected system will be more difficult to compromise).

For most formalisms, attack (defense) graphs (ADGs) are not reusable; one has to generate a new attack graph for each system. The Meta Attack Language (MAL) (Johnson et al., 2018), which this work is based on, was designed to enable the automatic generation of attack graphs from a system architecture model. The goal of this was to enable security assessments of system architectures. A key requirement in this development was that the assessment had to be probabilistic, as we face a lot of uncertainties when doing system-level security assessments.

It can be used to define domain-specific languages (DSLs) for different domains such as industrial control systems (ICSs), the Internet of Things (IoTs), vehicles, etc. Conceptually, a MAL representation of a system consists of different assets (e.g., Host), attack steps on the assets (e.g., Host.Compromise), defenses on assets (e.g., Host.EndPointDetection), and associations between different assets. All those parts comprise the MAL specification, defined in a MAL file. MAL attack graphs follow the standard attack defence logic, as described above. Moreover, if enabled, defenses related to attack steps act as obstacles for the connected attack steps.

A local time to compromise (TTC) probability distribution can be associated with each attack step, representing the difficulty (expressed in the time needed) of succeeding with that attack step. A combination of different probability distributions can be used for attack steps (e.g., a Bernoulli distribution multiplied by an Exponential distribution would mean that something is both uncertain and difficult to happen).[1]

Defenses on assets in coreLang (and in any other MAL-based DSL) are implemented as attack steps with a Bernoulli probability distribution associated with them, with the distribution's parameter configuring by the modeler. This practically implies that for a given parameter value p, the probability of the defense being enabled is p, and its probability of being disabled is $1-p$. Additionally, two discrete Boolean values can be selected as the parameter for defenses: "true" and "false". These values correspond to $p = 1$ and $p = 0$, respectively.

2.2. coreLang

Many ICT infrastructures share several components with similar behavior and properties, such as networks, computing units, and data, which motivated us to develop a language that covers these commonalities (Hacks and Katsikeas, 2021). coreLang[2] (Katsikeas et al., 2020) provides a series of predefined assets that offer a basic framework. Thus, it can serve as a starting point to create more domain-specific languages and be used to model generic environments. For a detailed background and motivation behind coreLang, the interested reader is referred to Katsikeas et al. (2020).

Modeling generally entails abstraction as it seeks to capture the relevant interactions between elements while discarding the extraneous aspects. Given coreLang's generalist scope, the abstraction level of the language is relatively high; the goal is to portray the essential behavior of components that exist in typical ICT infrastructures. This means that the assets of the language have to be generic and, therefore, do not capture specialized behavior regarding their functionality. For example, a database manager and a video processing application will look the same from coreLang's perspective as an `Application` that has `Data` associated with it, featuring the same types of inherent attack and attack vectors. However, the interactions between assets and enabling particular defenses can often capture the most salient facets relating to the specific purpose of an asset. A modeler could, for instance, specify that the database `Application` uses multiple `Data` assets as backup storage, by this changing the attack vector and making it more resilient to an attacker's disruptions.

[1] For a summary of the probability distributions available in MAL languages see https://github.com/mal-lang/malcompiler/wiki/Supported-distribution-functions.

[2] coreLang can be found on GitHub under the following link: https://github.com/mal-lang/coreLang.

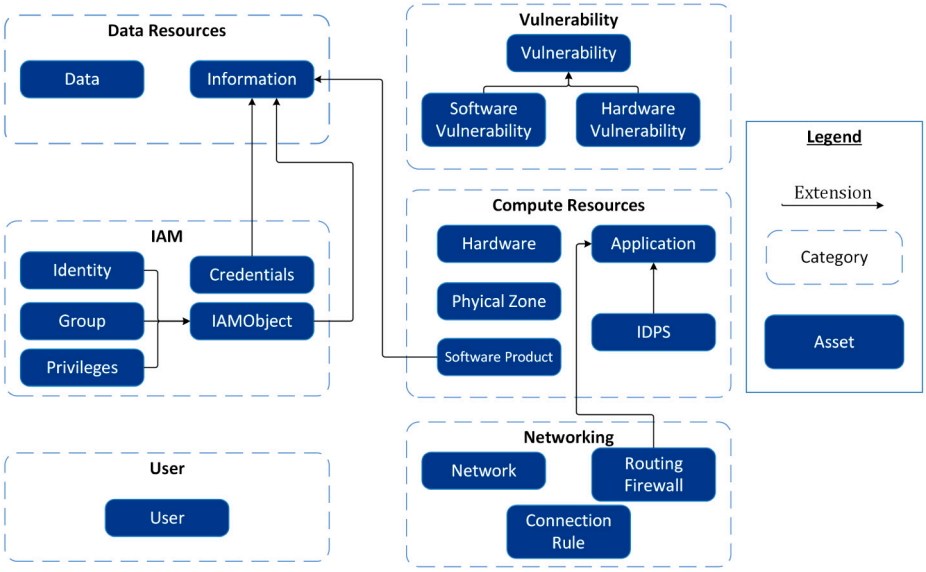


Fig. 1. Overview of coreLang.

The attack graphs generated by coreLang are cyclical, and it is important to note that the probabilities assigned to both attacks and defenses remain constant, irrespective of the attacker's actions. This means that our model does not incorporate a closed-loop scheme where the probabilities would adapt or change in response to the attacker's maneuvers. They represent a snapshot of the potential threats in the modeled environment. However, there are attack vectors modeled in coreLang that include bypass mechanisms where an attacker can perform the disruption with additional effort, even when the appropriate defenses are in place.

The language does not account for introducing new asset instances into the model as part of the attack process. For situations where inserting malicious components is essential to the operation, a speculative asset (an asset instance that is a hypothetical asset added to the model by the modeler to explore potential attack scenarios) could be used to represent the possibility that such an asset would be present and explore the attack vectors it would generate. It must also be noted that most of the probability distributions and their values assigned to the attack steps of coreLang have not been thoroughly researched yet. Hence, we see that as a point for future work.

Since coreLang is based on MAL, which uses a shortest path analysis for generating the attack graphs, it is focused on the mechanisms of a compromise rather than preparatory work preceding it, such as exploring networks for possible targets or identifying potential vulnerabilities in systems. The question that the attack graph generation process answers is, therefore, the following: given that the attacker is able to find all misconfigurations and vulnerabilities, and they could attempt all the attacks, which is the easiest path to reach a specific target?

Fig. 1 presents the overall structure of coreLang. Assets in coreLang are grouped into six categories: `Compute Resources`, `Vulnerability`, `Data Resources`, `User`, `IAM (Identity and Access Management)`, and `Networking`. Below, we briefly describe each of these six categories.

2.2.1. Compute resources

Software applications are the most common component of typical ICT infrastructure and with exceedingly rare exceptions, cyber attacks target one or more of them as part of their operations. As such, the `Application` asset, which represents any software running on a system, is the most important asset in the language. It also contains many attack steps to depict an attacker's various actions, and many other assets interact with it.

Analogously, the `Hardware` asset specifies the hardware systems on which `Applications` can run. This asset allows the attacker to obtain physical access to the hardware devices on which software applications are running.

`PhysicalZone` assets are used to group `Hardware` and `Network` assets to represent that they are housed within the same area. If an attacker gains physical access to a `PhysicalZone`, then they get physical access to all the `Hardware` and `Network` assets within it. Another asset is the `SoftwareProduct` to specify that multiple `Applications` use the same software package; the attacker can also use this to launch supply chain attacks that compromise the source of the software being used in a particular environment. Various defensive measures used to protect software applications coalesced into the `IDPS` (intrusion detection and prevention system) asset. An `IDPS` restricts many of the attacker's actions on the `Applications` it protects. Because the `IDPS` asset extends `Application` it is also susceptible to the same types of attacks normally targeting software applications. As such, an attacker may be able to disable an `IDPS` to prevent it from protecting the `Applications` associated with it.

2.2.2. Vulnerability

Vulnerabilities allow attackers to exploit flaws or weaknesses in software and hardware components to attain their goals. As the name implies, the `Software Vulnerability` asset is used to represent vulnerabilities in software components (i.e., `Applications`, `IDPSs`, or `SoftwareProducts`). `SoftwareVulnerability` was designed to integrate with the Common Vulnerability Scoring System (CVSS) version 3.1 framework (FIRST, 2024). A series of defenses on the `SoftwareVulnerability` asset are used to depict the CVSS exploitability and impact metrics, allowing the modeler to represent a wide variety of vulnerabilities. It must be noted that those defenses do not necessarily need to have a binary value; they can have any value between 0 and 1, according to the CVSS documentation. Table 1 shows an example of how a vulnerability's, CVE-2021-43890 (MITRE, 2024a),[3] CVSS 3.1 score translates into the defense values for a `SoftwareVulnerability` asset. `HardwareVulnerabilities` function similarly, but they are used in association with hardware systems and are less configurable. The distinction between `SoftwareVulnerability` and `HardwareVulnerability` in coreLang is

[3] CVE-2021-43890 describes a vulnerability in the Windows AppX Installer.

Table 1
CVSS v3.1 to coreLang mapping for CVE-2021-43890 (MITRE, 2024a).

CVSS metric	CVSS score	coreLang defense	Value
Attack vector	Network	NetworkAccessRequired	1
		LocalAccessRequired	0
		PhysicalAccessRequired	0
Attack complexity	High	HighComplexityExploitRequired	1
Privileges required	Low	LowPrivilegesRequired	1
		HighPrivilegesRequired	0
User interaction	Required	UserInteractionRequired	1
Scope	Unchanged	–	–
Confidentiality	High	ConfidentialityLimitations	0
Integrity	High	IntegrityLimitations	0
Availability	High	AvailabilityLimitations	0

based on where the impacts of the vulnerability are observed rather than where the flaw enabling it is found. For example, a hardware flaw that enables the attacker to exploit a particular operating system component would be represented as a `SoftwareVulnerability` on an `Application` depicting the operating system rather than a `HardwareVulnerability` on the flawed `Hardware` system.

2.2.3. Data resources

To represent data that can be stored on `Applications` or `Hardware` or transmitted between `Applications` over `Networks`, we created the `Data` asset. An attacker accessing the data can then potentially read or alter it. `Information`, the other asset in this category, is more conceptual than most other assets. It represents any information, and since information is immaterial, the `Information` asset always needs to be associated with `Data` containing it for it to be accessible for potential attacks. `Data` replication is also implemented by specifying that the same `Information` is backed up across multiple replicas.

2.2.4. IAM

Identity and Access Management (IAM) plays an important role in cyber-attacks, where the adversary gains access to systems through legitimate authentication mechanisms by utilizing existing accounts and/or modifying their associated privileges. The first set of assets belonging to this category, `Identity`, `Group`, and `Privileges`, deal with this aspect. As shown in Fig. 1, all three of these assets extend the abstract `IAMObject` asset; this is because they function very similarly in that they define access permissions to various `Applications` and `Data`. In many circumstances, any of the three could be used, and we will refer to all three collectively as the `IAMObject` assets in those cases. `Groups` can be used to hierarchically organize `Identities`.

The other asset in this category is `Credentials`, which can represent any token used in various authentication mechanisms, from physical ones, such as key cards or retina scans, to digital ones, like passwords and cryptographic keys. Due to its eclectic nature, the `Credentials` asset can be configured to match the required behavior.[4] In addition to serving as a means of authenticating accounts through an association with an `Identity`, `Credentials` can also be used to encrypt or sign `Data`. Because `Credentials` extend `Information`, they possess all of its attacks and defenses. As such, an attacker that can reach `Data` that contain `Credentials` can use them to decrypt, spoof, or authenticate.

2.2.5. User

The User category consists of the `User` asset, which acts as an attack surface for different social engineering attacks. Usually, an `Identity` belongs to a `User`, and a `User` can have multiple `Identities` associated with it. The attacker can either extract credentials via phishing or trick the user into performing malicious actions (e.g., tricking the user into clicking on a malicious link). The security awareness of a user is accounted for in a defense that reduces the likelihood that attacks targeting the user would succeed.

2.2.6. Networking

Networking assets are typically used to extend an attacker's reach through the modeled environment. coreLang's approach to networking is to provide a generic protocol-agnostic way of establishing connections. The `Network` asset abstractly covers all seven layers of the OSI stack. It can be used to bridge the communications between multiple `Applications` and can carry `Data` that are in transit over it. Usually, an attacker that succeeds in gaining access to a `Network` can connect to the associated `Applications` and access the `Data` in transit over the `Network`.

The `ConnectionRule` asset was introduced to represent network connections permitted by firewall rules. `ConnectionRules` are used to define allowed communications between `Application` assets and/or `Network` assets. A frequent pattern seen in models is having an `Application` reach a `Network` via a `ConnectionRule`. Directionality must also be specified for `ConnectionRules`, ingoing, outgoing, diode, or bidirectional. The `RoutingFirewall` asset can be used to manage `ConnectionRules`, if it compromised[5] the attacker can bypass the restrictions on existing `ConnectionRules`.

2.3. Related research

A large body of work can be found in the literature regarding threat modeling in general (Xiong and Lagerström, 2019). However, most works focus on threat modeling techniques and/or frameworks. Examples include STRIDE (Shostack, 2014), which helps discover threats during product or system design, while approaches such as UMLsec (Jürjens, 2002, 2005), SecureUML (Basin et al., 2006, 2011), SECTET (Alam et al., 2007; Hafner et al., 2006), or STS-ml (Paja et al., 2015) enable a more model-based security analysis. Other model-driven approaches rely on attack trees and/or graphs (Schneier, 1999; Phillips and Swiler, 1998; Mauw and Oostdijk, 2005; Kordy et al., 2010, 2014) as discussed earlier.

Instead, the presented work is perhaps best related to works that focus on validating the use of threat modeling methods or frameworks, for example, as part of case studies, such as the ones that have been covered by the literature review performed by Tuma et al. (2018). More specifically, related work investigating the feasibility or validating attacks or DSLs is of particular interest.

Tayouri et al. (2023) estimated the attack scenario coverage of MulVal (a generic framework for constructing logical attack graphs), by mapping all MulVal interaction rules to ATT&CK Tactics and Techniques. They found MulVal rules to cover less than a quarter of the ATT&CK Techniques. They called for further research into attack graph generation frameworks that are more complete and have higher attack scenario coverage.

In the following, we discuss related works in the domain of MAL that performed some validations. Åberg and Sparf (2019) showed that MAL is effective by randomly selecting 11 attack techniques from the MITRE ATT&CK matrix and successfully implementing them in the language. vehicleLang, a threat modeling and attack simulation language specifically designed for the automotive domain (Katsikeas et al., 2022), has been validated in van der Schoot (2020) and Skobalj (2019). The goal of these validations was to measure the real-world applicability of the language, and the method involved the creation

[4] It can be set as guessable (e.g., via brute-force), unique, which means it cannot be reused, or phishable, which would allow an attacker to obtain them via social engineering.

[5] The `RoutingFirewall` extends the `Application` asset; therefore, it can be taken over by the attacker like any regular software application.

of models from vehicles to perform threat modeling. Then, a more advanced validation, using domain experts participating in a Turing-based test, has been performed (Katsikeas et al., 2022). Nordgren and Sederlin (2021) validated the correctness and effectiveness of a DSL called enterpriseLang. The work used the language to model and simulate three real-world cyber attacks and performed a comparative study. However, none of the validation works performed before was about coreLang, and none of the previously referred works is of the same detail and thoroughness as the validation performed in this.

Xiong et al. (2022) looked closely at enterprise systems and proposed a way to extract adversary technique information from the ATT&CK enterprise Matrix and integrate and map it into enterpriseLang.

Hacks et al. (2021) attempted to integrate security behavior information into attack simulations. This was demonstrated through a two-step mapping, i.e., first from Security Behavior Analysis (SBA) to MITRE ATT&CK and then from MITRE ATT&CK techniques to a MAL-based DSL named icsLang (which is based on an earlier version of coreLang). This work aimed to make attack simulations more accurate and realistic.

Pirca and Lallie (2023) performed an interesting empirical study by evaluating the effectiveness of attack graphs and MITRE ATT&CK in aiding cyber-attack perception after an attack has taken place. They effectively managed to identify that attack graphs were not only preferred by participants, but the method proved better at aiding cyber attack perception among them.

To summarize, while there exists some limited work looking at the validation of other DSLs and related frameworks and creating a mapping between ATT&CK (MITRE, 2024) and DSLs, as described above, to the best of our knowledge, no existing work has considered validating coreLang specifically, with the only exception of our own, just published, evaluation of coreLang against domain experts and student hacking performance (Katsikeas et al., 2024), in which we empirically evaluated the simulation results (that take the form of attack paths) of coreLang against the assessments of humans, effectively trying to answer the question whether those attack paths seem to be identified in a well balanced and correct manner. This research gap in validating coreLang motivates the need for this work and the fact that the validation is done against a comprehensive collection of different real-world attacks for the same domain as for which coreLang has been designed for, adds a higher value to the validation.

3. Method

For this work, we rely on the Design Science Research (DSR) principle (Hevner et al., 2004) to design an artifact that seeks to enhance human knowledge. DSR is a form of scientific knowledge production that involves the development of innovative constructions intended to solve problems facing the real world and simultaneously makes a scientific contribution. An important outcome of this type of research is an artifact that solves a domain problem. DSR is based on an iterative approach, which allows researchers to rely on multiple approaches, including incorporating end-users' and stakeholders' perspectives. The iteration process allows researchers to revisit and refine research results and address the explicated problem.

Over time, various interpretations of DSR have been developed, and Venable et al. (2017) compare DSR methodologies and put forward a framework to help choose the best fitting approach. Specifically, they differentiate between objectivist, positivist methodologies and subjectivist, interpretive methodologies. They argue that if one expects the designed artifact to be the best solution for a generalized target group that behaves the same, one should opt for one of the objectivist, positivist methodologies, which aligns with our expectations towards the created artifact, i.e., coreLang.

Furthermore, Venable et al. (2017) also state that "If extensive adaptation to daily use is needed, then choose design science research methodology (DSRM) (Peffers et al., 2007)". As we foresee coreLang to be used regularly by the wider community, both to develop new languages and as a language for threat modeling itself, we align our research method with the six activities of Peffers et al. (2007),[6] as in the original work (Katsikeas et al., 2020). Specifically, our focus is on the design cycle iterating between core language development (i.e., design phase (Peffers et al., 2007)) and the validation of the work (i.e., evaluation phase (Peffers et al., 2007)). The problem description, description of objectives, and a first demonstration of coreLang are presented in the first article (Katsikeas et al., 2020).

An important part of designing and validating the language was weekly brainstorming meetings with domain experts and practitioners that involved academics and industry people. In those meetings, each of the participants had certain roles; for example some of them were performing validation work, others were acting as reviewers, while we were acting as the main developers and project managers of the development process. The goal of these conversations was to ensure that coreLang reflects behavior in line with what is expected from common ICT infrastructure. Examples of frequently discussed topics were how to adequately represent particular adversary activities in the language and conversations regarding how very specific parts of modern ICT infrastructures can be modeled and used as input for the validation presented in this work. These meetings took place with different external participants over two years since participating people fluctuated.

3.1. Process of validation against MITRE ATT&CK

To validate coreLang we required a broad collection of attacks to verify if they could be implemented in the language. The MITRE ATT&CK Enterprise Matrix (MITRE, 2024b) version 12.1,[7] hereafter referred to simply as the matrix, is a curated database that contains a large number of techniques that an attacker could employ as part of their operations. We selected it for several reasons: it offers a comprehensive collection of procedures, although not exhaustive; its generic nature and level of abstraction are comparable to coreLang's, and it includes mitigations to counter the adversary activities described. The matrix has also garnered sufficient widespread usage in the cyber-security domain (Tayouri et al., 2023; Derbyshire et al., 2021) that it serves a convenient framework that many will be familiar with.

The matrix is divided into 14 tactics that are used to group techniques together according to the attacker's goals in employing them; some techniques are included in multiple tactics. The tactics cover different facets of an attacker's actions at various points of the attack operation's lifecycle. Based on the design choices and assumptions previously discussed, coreLang does not cover these dimensions or phases equally well. As the following section will explain, on a tactic-by-tactic basis, some are a particularly good fit for the language, others are not reflected in coreLang at all, while others are partially represented. For our work, each technique was evaluated separately within the context of a specific tactic, according to the aspects of the technique on the said tactic. This means that the same technique could be scored as being covered well by coreLang in one tactic, while being found lacking when evaluating it within the context of different associated tactics.

MITRE ATT&CK describes their abstraction level as "mid-level adversary modeling" (Strom et al., 2018), which generally fits well with coreLang's goals and own level of abstraction. Additionally, the tactic, technique, and sub-technique hierarchy give us more options to select the abstraction level that best suits coreLang's conceptualizations. Techniques are presented as the natural evaluation unit, since they describe individual actions that an adversary may pursue to achieve their goals.

[6] Identify Problem, Define Objectives, Design Artefact, Demonstration, Evaluation, Communication.

[7] https://attack.mitre.org/resources/updates/updates-october-2022.

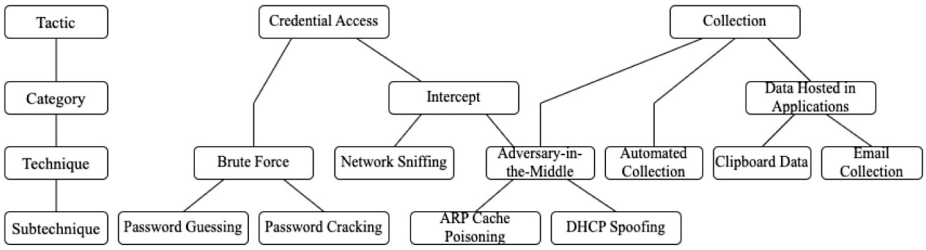


Fig. 2. Augmented ATT&CK hierarchy.

Sub-techniques are typically too specialized for our purposes as they deal with specific technologies or implementations, and coreLang's assets lack specialization. However, in some cases, even techniques proved too detailed. In those situations, we introduced a new layer between tactic and technique that we labeled *category* that groups together multiple techniques conceptualized similarly in coreLang. Fig. 2 depicts what the ATT&CK hierarchy looks like after the addition of the category level that we introduced. As an example of a category introduced, that is also present in the figure, in the *Collection* tactic we grouped the *Clipboard Data* and *Email Collection* techniques under the *Data Hosted on Applications category* since both are represented as gaining `Read` on a `Data` asset hosted on an `Application`. The extent of the coalescence varies due to the uneven abstraction in both ATT&CK and coreLang. Some tactics are dominated by one or two broad *categories* that encompass most of the techniques within them. Still, no tactic covers all its techniques by a single *category*.

The matrix also contains a collection of mitigations associated with the (sub-)techniques they counteract. This helped us to design and evaluate the countermeasures present in coreLang by verifying that coreLang's defenses limited the impact or likelihood of certain attacks. We discuss how mitigations are addressed in the language in Section 4.8.

3.2. Validation mapping

The validation process involved going through the 14 tactics in the matrix and evaluating how well the language covered the comprising techniques. Some tactics are not meaningfully accounted for in coreLang (cf. 2), and an explanation for why they are not represented in the language is given in Section 4.11. However, in some cases, not all aspects of a technique are covered, and we discuss what is missing and the reason. For the tactics that the language addresses either entirely or significantly, the description of the coverage is complemented by a mind map to provide an overview of how the various techniques are represented in coreLang and highlight the *categories* we have introduced to group several techniques together. To further clarify how the mechanisms of the tactic are depicted in coreLang, one or two representative techniques or categories are given a more detailed explanation.

Each mind map is structured around a root node that always bears the name of the tactic and only serves as a central connection point for other nodes. All of the leaf nodes depict a specific technique or sub-technique; their outline indicates how well they are covered by coreLang. Usually, sub-techniques are not included in the mind maps since they are too specific for coreLang. However, in a few circumstances, sub-techniques provided the best mapping to the behavior of the language, and in those cases, they are explicitly pointed out. Where we grouped several techniques under a *category*, it shows up as an intermediary node between the root and leaves. The following paragraphs will explain how the techniques (or sub-techniques) are classified according to how well they are represented in the language.

Technique is explicitly represented in the language

A *technique* is marked as explicitly represented when an attack step, association, or asset can be found in coreLang that directly depicts that concept. For example, the *Endpoint Denial of Service* technique that is part of the *Impact* tactic refers to an attacker's ability to compromise the availability of a particular software application. In coreLang terms, this directly translates to the attacker being able to trigger the `Deny` attack step on an `Application` asset.

Technique is more specific than coreLang representation

When we introduced a *category* to group multiple techniques, some of the encompassed techniques described behavior that was too detailed for coreLang but conceptually lined up with its parent *category*. This means that the techniques' disruptions are represented in coreLang, but their implementation-specific facets described by ATT&CK are not reflected in the language.

As an example, the *System Shutdown/Reboot* technique in the *Impact* tactic is described as a more specific variation of the *Application Denial of Service category*[8] in our *Impact* mind map. This is because the operating system is represented in coreLang by an `Application` asset. Therefore, it fits the general pattern defining the category of the attacker successfully reaching the `Deny` attack step trigger on the target `Application` asset. Still, the language does not cover that it is specifically *shut down*.

Technique is not adequately covered

Some techniques cannot be represented in coreLang because the language does not cover some of their distinguishing features. These techniques will not appear in the mind maps and are not usually discussed when validating the tactics. The Discussion, Section 5, explains why certain types of techniques do not map to the language.

To help the validation process, we produced several models[9] that describe how particular techniques or *categories* are implemented in coreLang. They can be found in the "validation" folder of the example models repository (Buhaiu et al., 2022). Many explanations of how techniques and categories are represented in coreLang will include a figure containing a picture of the model and an attack graph of how the attacker can reach the relevant attack step.

Because the software used to generate the attack graphs hides certain attack steps to reduce the number of nodes for legibility purposes, some of the attack steps mentioned in our descriptions may not show up in the attack graph figure. Therefore, when an explanation refers to an attack step that is not visible in the attack graph, it is either hidden or a prefixed version is shown instead.

[8] For a more detailed explanation of the *Application Denial of Service category* see Section 4.4.1.

[9] These models were created using the securiCAD Professional tool provided by the company Foreseeti. However, the company and the tool no longer exist. Cf. e.g. https://web.archive.org/web/20220813034402/https://foreseeti.com/.



Fig. 3. coreLang coverage of the MITRE ATT&CK matrix (v12.1).

Table 2
coreLang coverage of the MITRE ATT&CK tactics.

ATT&CK tactic	Coverage (of techniques)
Reconnaissance	Not covered
Resource Development	Not covered
Initial Access	7 out of 9
Execution	13 out of 13
Persistence	Not covered
Privilege Escalation	13 out of 13
Defense Evasion	21 out of 43
Credential Access	12 out of 17
Discovery	Not covered
Lateral Movement	7 out of 9
Collection	14 out of 17
Command and Control	Not covered
Exfiltration	4 out of 9
Impact	13 out of 13

A technique or *category* flagged as modeled denotes that a model exists for it in the repository; Figures of several models have been included in the explanations. Typically, the nodes directly connecting to the root, either *categories* or techniques not grouped under a *category*, are modeled. Usually, if a *category* is modeled, then the techniques it subsumes are not since the model for the *category* applies to all of the comprising techniques. When this is not the case, the description of the *category* or techniques explains the reason.

The scenarios described in many techniques involve reaching a particular system and performing certain actions. We omit the steps preceding the actions pertinent to the techniques discussed to simplify the models and focus on the relevant aspects. As such, we assume that the attacker has already reached a certain stage in the attack operation and continues from that point onward. In practice, we often connect the `Attacker` directly to the asset where the important actions are taken and exclude assets the attacker would first need to compromise to get to those actions. A few techniques that explicitly deal with reaching those initial systems in the first place, most notably in the *Initial Access* tactic, will have models that also include these assets and attack steps. The values for defenses on assets in the models have been left to default values unless otherwise stated.

4. Validation

In the subsections that follow, a discussion of how coreLang is mapped against the tactics, techniques, and sub-techniques of the ATT&CK matrix is provided. The ATT&CK techniques, sub-techniques, as well as the categories, that we introduced, are stylized using *this font*, while coreLang elements, such as assets, attack steps, defenses, and associations will be stylized using `this font`. Finally, coreLang model instance names will use the sans serif font family. A complete example would then be: "Performing an *Input Capture* on the Firefox `Application`".

Table 2 showcases coreLang's coverage of all the available tactics in the matrix, while Fig. 3 depicts all the matrix techniques that are explicitly covered (marked with dark green color) and the techniques that are covered but are more specific than coreLang's representation (marked with light green color).

4.1. Initial access

The *Initial Access* tactic consists of techniques that adversaries use as entry vectors to gain their initial foothold within a target network. It includes nine techniques, some of which we have grouped under the *Exploit Accounts* category. Fig. 4 presents the techniques organized as a mindmap for this tactic.

4.1.1. Exploit public-facing application

This technique deals with the attacker gaining access to a system that is reachable via the Internet by exploiting a vulnerability.

In Fig. 5, the attacker is connected to the `Internet` represented by a `Network` asset, and from there, they can establish a network connection to the `Target Application`, represented by the `NetworkConnectUnispected`[10] attack step.

Compromising an externally accessible system is a frequent initial step in more complex attacks. When addressing those, we just grant the attacker a network connection on the relevant asset(s) via `NetworkConnectUninspected` to keep the attack vectors and models smaller and easier to read. We implicitly assume that the attacker could perform these steps as described for this technique.

Once the attacker can connect to the `Target Application`, they can exploit any `SoftwareVulnerabilities` associated with it. For a more detailed description of how `SoftwareVulnerabilities` can be used to compromise `Applications`, see Section 4.2.1 in the *Execution* tactic.

4.1.2. Exploit accounts

External Remote Services, *Trusted Relationship*, and *Valid Accounts* techniques all rely on the attacker leveraging existing accounts that they have gained access to on different systems. User accounts are represented in coreLang by the `Identity` asset. Suppose an attacker can assume an `Identity`. In that case, they can authenticate on the `Applications` and access any `Data` according to the privilege associations of the `Identity`. The example model in Fig. 6 shows

[10] `NetworkConnectUnispected` means that the attacker can connect to the system and the communication is not impeded by any payload inspection mechanisms (The `Network Intrusion Prevention` paragraph in the Mitigations Section 4.8 explains how payload inspection mechanisms are modeled in coreLang.).

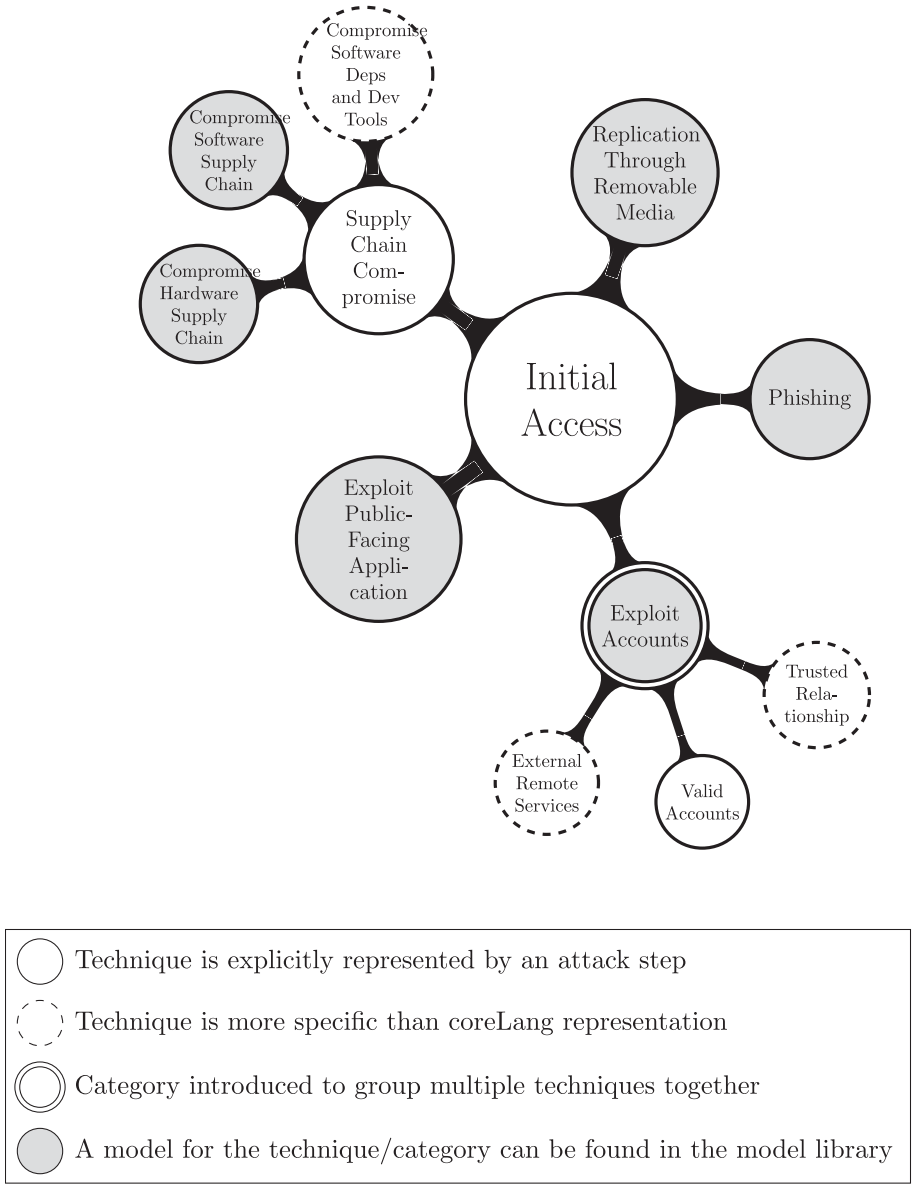


Fig. 4. Initial access mindmap.

how the attacker can compromise the target `Application` since they can `Authenticate` on it using the `ExecutionPrivilegeAccess` granted by the `Identity` they have assumed.

4.1.3. Phishing

Phishing encompasses different actions the attacker takes that trick the user into inadvertently granting access. It is represented in coreLang via the `SocialEngineering` attack step on the `User` asset. `SocialEngineering` has two parts, `CredentialTheft` and `UnsafeUserActivity`. `CredentialTheft` allows the attacker to utilize the `Credentials`.[11] belonging to `Identity` assets associated with the `User` This can, however, be prevented if the `NotPhishable` defense on the `Credentials` is enabled, which represents the fact that they cannot be phished in the case they are biometric, e.g., fingerprints or iris scans, or physical, e.g., access cards or keys. `UnsafeUserActivity` represents the user being induced to unwittingly run malicious code[12] on their systems. Because both stealing credentials and tricking the user into running malicious code are hard to predict accurately and may require several attempts, the `CredentialTheft` and `UnsafeUserActivity` attack steps have probabilistic distributions assigned to them that reflect this additional time and effort required. The model shown in Fig. 7 contains the basic scenario for a phishing attack.

As explained in Section 4.8 Mitigations, the `securityAwareness` defense on the `User` asset reduces the likelihood that the social engineering actions of the attacker are successful. However, this defense is not infallible, and the attacker may be able to bypass it, as will be discussed in Section 4.9 Defense Evasion in more detail.

4.1.4. Replication through removable media

Replication Through Removable Media is implemented via the entry point attack step `AttemptDeliverMaliciousRemovableMedia`

[11] Section 4.5 Credential Access details how the attacker can use `Credentials` in their attacks.

[12] The *Application Code Execution* category, Section 4.2.1, in the `Execution` tactic discusses how execution of malicious code is represented in the language.

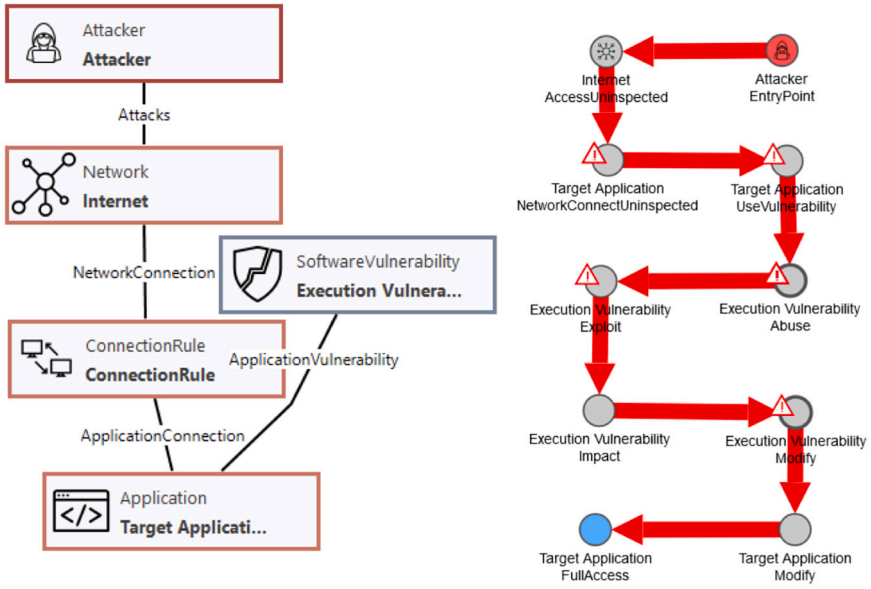


Fig. 5. *Exploit Public-Facing Application* attack in coreLang.

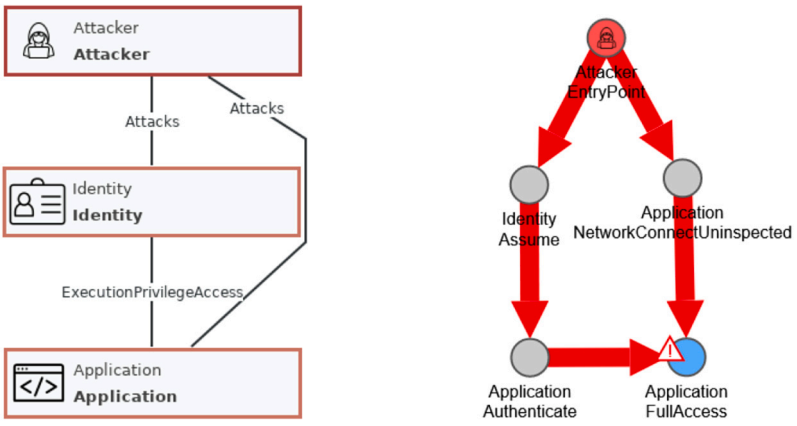


Fig. 6. *Exploit Accounts* attack in coreLang.

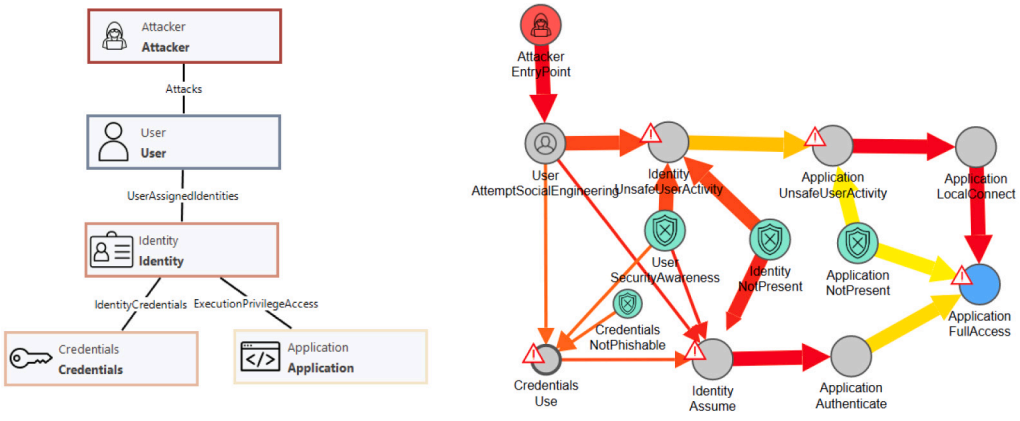


Fig. 7. *Phishing* attack in coreLang.

on the `User` asset. This attack step propagates via the `PhysicalZone` and `Hardware` assets to reach `Applications` running on `Hardware` that this particular `User` has access to. It then seeks to use vulnerabilities or the user's privileges on those `Applications`. The aforementioned `securityAwareness` defense also influences the likelihood that this attack is successful. Additionally, the `noRemovableMediaUsage` defense on the `User` asset completely prevents it as the `User` is not expected ever to use removable media and is therefore not a viable vector. Lastly, the `hardwareModificationsProtection` defense on the `Hardware` asset functions in

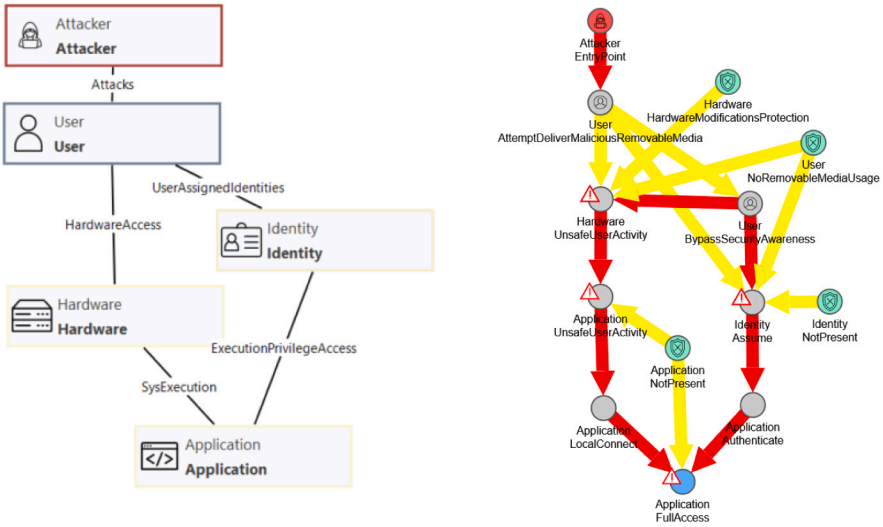


Fig. 8. *Replication Through Removable Media* in coreLang.

a more localized fashion and prevents the attack from reaching the `Applications` running on it. The model shown in Fig. 8 contains the basic scenario of a malicious removable media delivery attack.

4.1.5. Supply chain compromise

The *Supply Chain Compromise* technique is one of the situations where its sub-techniques are better matching for coreLang than the parent technique. The *Hardware Supply Chain Compromise* is implemented via the `SupplyChainAttack` step on the `Hardware` asset. If successful, `SupplyChainAttack` leads to `FullAccess` on the `Hardware`. Because hardware supply chain attacks are very advanced and complex, the attack step has a probability distribution considering the significant effort required. The `supplyChainAuditing` defense on the `Hardware` asset prevents the attack. The *Compromise Software Dependencies and Development Tools* and *Compromise Software Supply Chain* sub-techniques can be represented similarly in coreLang. They are implemented via the `Data` asset. A `SoftwareProduct` can be associated with a `Data` asset that serves as the source code of the software product; thus, if an attacker can `Write` said `Data` asset they can perform a software supply chain attack, which grants them `FullAccess` on all of the `Applications` that use that `SoftwareProduct`. The `supplyChainAuditing` defense, which is analogous to the one on `Hardware`, prevents the disruptions of the attack on the `Application` asset. This defense is set on each `Application` individually, rather than the `SoftwareProduct`, to represent the auditing process on the side of the organization making use of the software rather than the provider.

Firmware is particularly interesting since it straddles the line between software and hardware. Suppose the specific supply chain attack on firmware relates to pre-flashed firmware that is part of the initial hardware delivery. In that case, we have decided to represent that as a hardware supply chain compromise. If the attack relies on updates meant to occur on-site, the same logic as software supply chain attacks should be used.

The model in Fig. 9 showcases *Compromise Hardware Supply Chain* attack, while Fig. 10 covers a simple compromise of a `SoftwareProduct`'s `Data`.

4.2. Execution

The *Execution* tactic deals with attacker actions that grant them the ability to execute code on the targeted systems and, as such, is one of the most relevant ones since most cyber attacks will involve one or more of the techniques detailed within it. A visual representation of the coreLang mapping can be seen in Fig. 11.

4.2.1. Application code execution

The ability to execute code on a system is in coreLang portrayed as gaining `FullAccess` on an `Application`, which, as the name implies, fully compromises it. This is typically achieved by connecting[13] to an `Application`, and either abusing an existing `SoftwareVulnerability` or authenticating using an `IAMObject` (e.g. `Identity`) asset. Many of the techniques in this tactic fit these scenarios and were therefore grouped together under the *Application Code Execution* category. The encompassed techniques all conform to this general description and only differ in the purpose of the `Application` and/or the specific nature of the `SoftwareVulnerability`.

For the *Exploit Existing Accounts* category, Section 4.1.2, already explained how an attacker can `Authenticate` using valid user accounts and the model in Fig. 12 includes the `SoftwareVulnerability`, either of which is sufficient for the attacker to execute code on the Target Application once they connect to it.

Looking first at using legitimate privileges to get execution permissions, if an attacker can assume an `Identity` that either provides `HighPrivileges` or `ExecutionPrivileges` on an `Application`, then they can authenticate on that `Application` and simply run the code as they have the requisite permissions. Alternatively, if the `Application` targeted has a `SoftwareVulnerability` that impacts its integrity, i.e., the `SoftwareVulnerability` does not have a high `integrityImpactLimitation` defense value, the attacker is then able to fully compromise the `Application` by exploiting the vulnerability.

4.2.2. Application context Hijacking

The *Application Context Hijacking* category, which includes the *Container Administration Command* and *Software Deployment Tools* techniques, is fairly similar to the larger *Application Code Execution*. It describes situations where the attacker can utilize the security context of a system they have compromised to run code on other systems that the original context has permission for. The attacker's goal is still to gain `FullAccess`, representing arbitrary code execution, on an `Application`. Another resemblance is the fact that the attacker can attain the `FullAccess` via an `IAMObject` asset providing `HighPrivileges` or `ExecutionPrivileges` as was the case in one of the two situations presented above.

[13] Either by reaching the `NetworkConnectUninspected` as described when discussing the *Exploit Public-Facing Application* in Section 4.1.1 or `LocalConnect` attack steps.

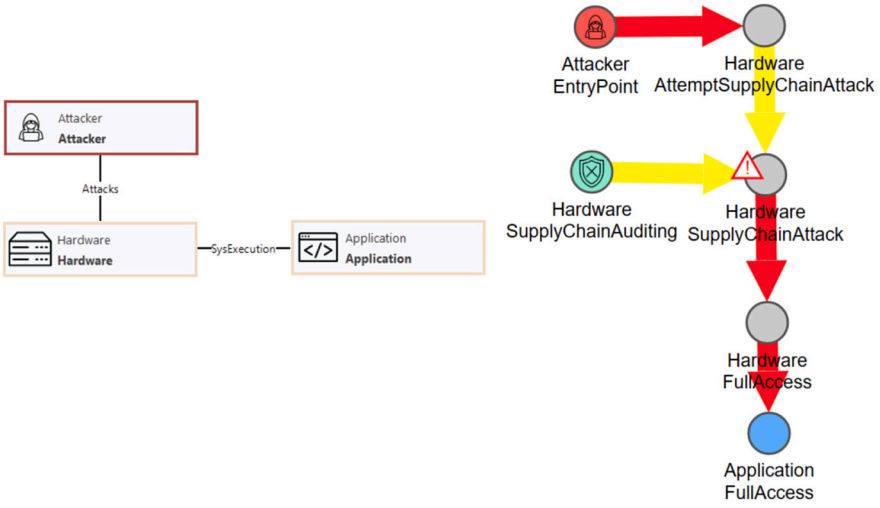


Fig. 9. *Compromise Hardware Supply Chain* attack in coreLang.

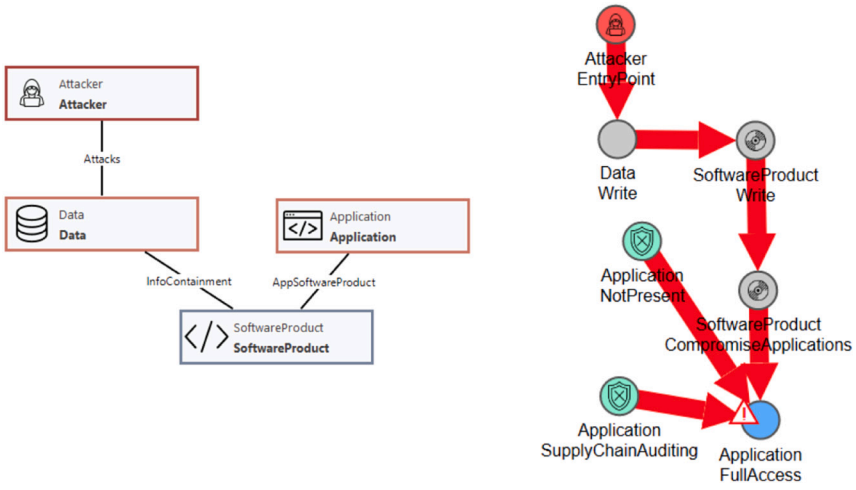


Fig. 10. *Compromise Software Supply Chain* attack in coreLang.

However, this category defines more specific requirements regarding what that `IAMObject` is and how the attacker can assume it. The easiest way to represent a security context in coreLang is by using the `Privileges` asset[14] that has `ExecutionPrivilegeAccess` on an `Application`. As shown in Fig. 13, if the attacker can reach the `FullAccess` attack step on the Initial Compromise(Compr) `Application`, the assumption is that they can take any actions within the *Security Context* `Privileges`. Since the *Security Context* provides `ExecutionPrivilegeAccess` to the Target Application, the attacker is successful in executing their code on it, gaining `FullAccess`. *Container Administration Command* and *Software Deployment Tools* define specific security contexts that can easily be modeled in the fashion just described.

4.2.3. Inter-process communication

To adequately represent the *Inter-Process Communication* (IPC) capabilities conferred to an `Application` by the operating system, coreLang makes use of the `Privileges` asset. For implementing IPC in coreLang, we envisioned the situation where the IPC capabilities allow one process to send execution commands to another, which is represented via a `Privileges` asset that has three types of associations:

- `ExecutionPrivileges` to the `Application` that can send the execution commands.
- `LowPrivileges` to the `Application` that represents the Operating System (OS). This propagates local connectivity via the OS to the receiving `Application`.
- `HighPrivileges` to the `Application` that can receive the execution commands.

Such a scenario is depicted in Fig. 14.

4.3. Collection

Many techniques described in this tactic involve collecting data from various sources. The `Data` asset is used in coreLang to represent different types of data which can be hosted on an `Application`, in transit over a `Network`, or stored on `Hardware`. The specifics of how the `Data` are stored or transmitted are immaterial to the language; only their presence is accounted for. As seen in Fig. 15, many of the techniques in this tactic can be grouped together and involve simple asset interactions. The *Exfiltration* tactic discussed later, which is logically linked to *Collection*, outlines more complex and nuanced language features.

[14] This is the case unless a specific access control role is logically tied to that context, in which case either an `Identity` or `Group` can be used, as appropriate.

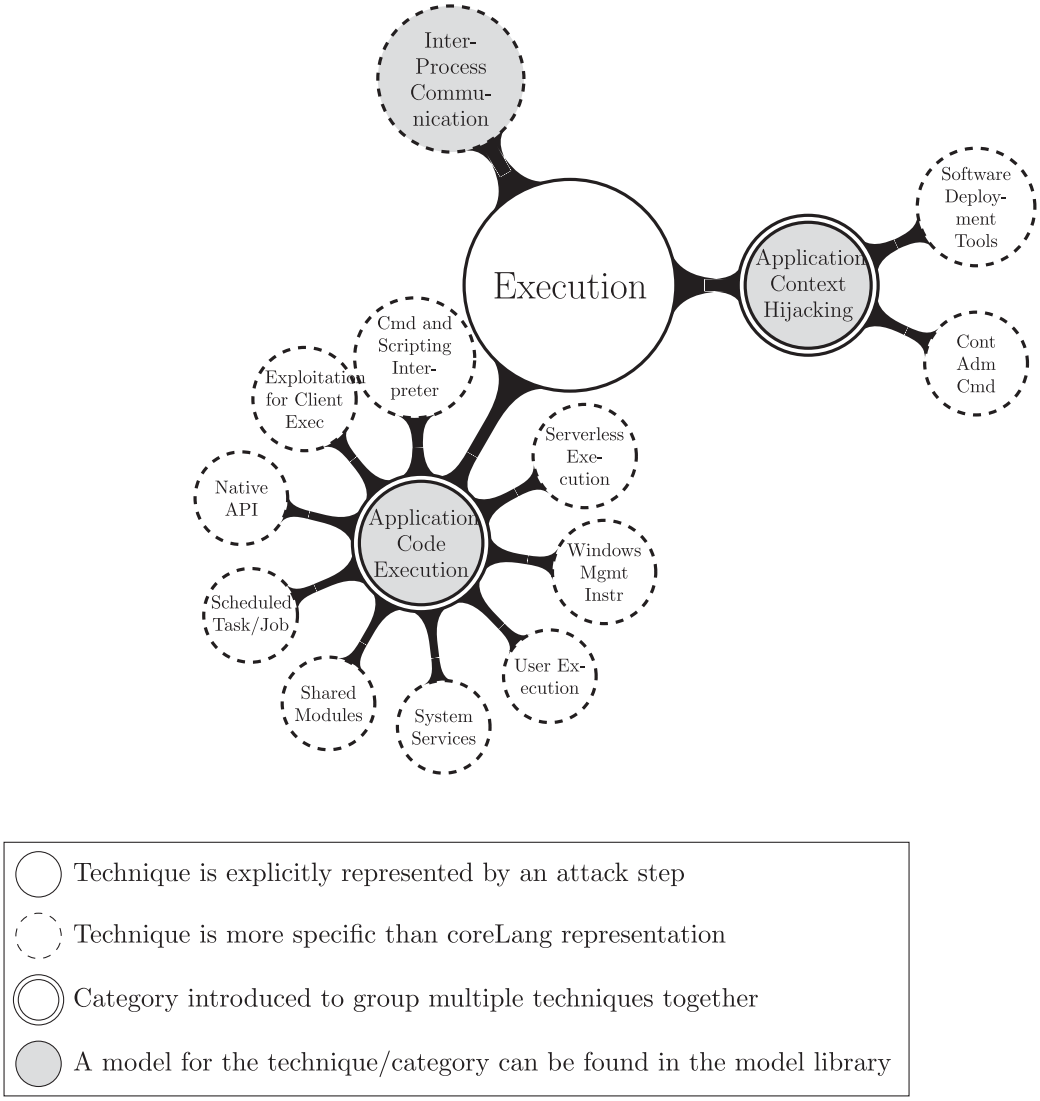


Fig. 11. Execution mindmap.

4.3.1. Data hosted on applications

The largest category in this tactic is called *Data Hosted on Applications* and, as the name implies, coalesces all of the techniques that describe obtaining data hosted on software applications. The various techniques that comprise it refer to different locations and contexts where the data can be found, all of which can be portrayed using `Applications` that fit the scenario descriptions.

Three distinct approaches exist regarding how the attacker can compromise the `Application` storing the `Data`. The first two of these are related to gaining access on an `Application` using a `SoftwareVulnerability` or `Identity`, as described in Section 4.2.1 *Application Code Execution*, while the last one leverages social engineering, as described in Section 4.1.3 *Phishing*, to gain access to the `Application` storing the `Data`. Fig. 16 shows the first approach; the attacker can exploit a `SoftwareVulnerability` in the `Application`. This `SoftwareVulnerability` then allows the attacker to `Read` the `Data` stored in the `Application`.

The second approach is shown in Fig. 17. In this case, the attacker has successfully attained the permissions, expressed via an `Identity`, required to log in to the `Application` legitimately. The attacker then uses the permissions granted by the `Identity` to `Read` the `Data`.

The third approach is shown in Fig. 18. The attacker can compromise the `Application` by performing `SocialEngineering` on a `User` that has access to it. This is then similar to the scenario where the attacker was able to obtain legitimate permissions because `UnsafeUserActivity` is interpreted as the attacker tricking the `User` into exposing their current security context, denoted by the `Identity` in this case.

Distinctions exist between the various techniques that have been grouped in this category. Still, they all relate to the modeling process and not to explicit coreLang attack steps or defenses. Different constellations (such as `Application` hierarchies) and configurations (specific `SoftwareVulnerabilities`) of assets can be represented to fully cover these scenarios, which eventually lead to a particular `Application` that stores the `Data` being compromised. For more details about how the attacker would be able to extract `Data` stored on an `Application`, see Section 4.10 Exfiltration.

4.3.2. Data stored on hardware

The *Data Stored on Hardware* category includes four techniques, *Audio Capture*, *Input Capture*, *Screen Capture*, and *Video Capture*, which all deal with extracting the `Data` from peripherals or hardware components. Suppose the `Data` can be obtained from the `Hardware` directly

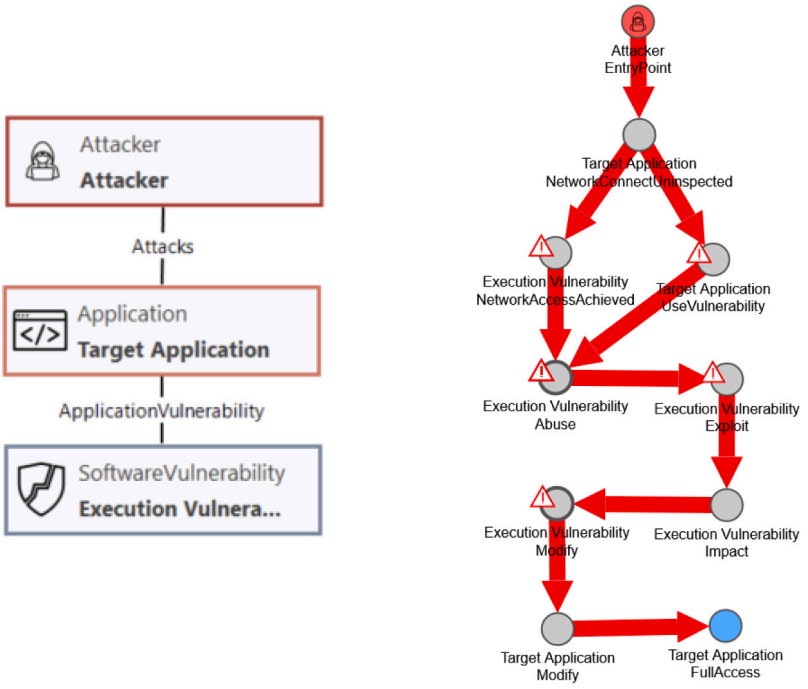


Fig. 12. *Application Code Execution* via `SoftwareVulnerability` in coreLang.

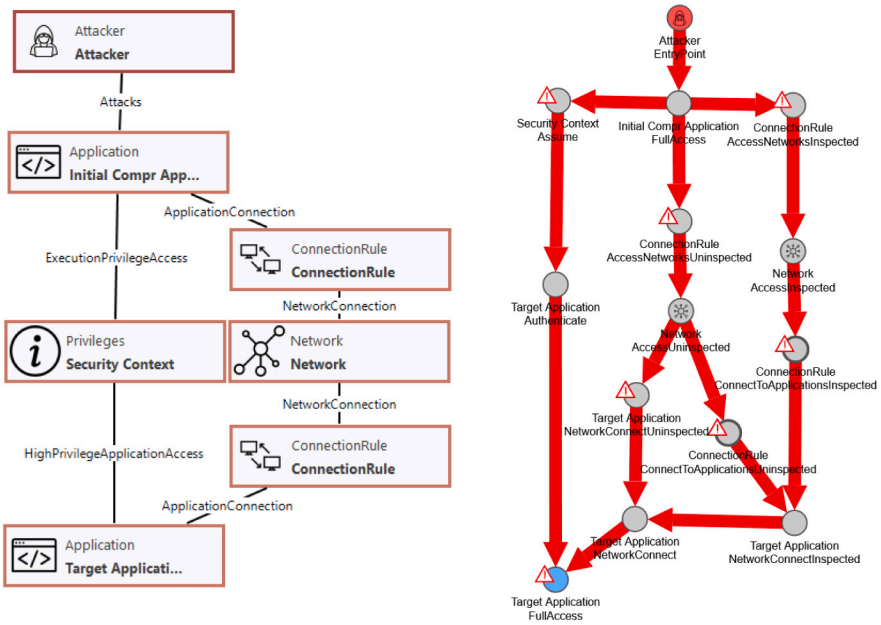


Fig. 13. *Application Context Hijacking* in coreLang.

without interfacing with any software, then it can be stored directly on `Hardware`. Each technique also details scenarios in which the data are extracted from the software applications using the hardware. They behave according to the *Data Hosted in Applications* description in those circumstances.

4.3.3. Adversary-in-the-middle

The *Adversary-in-the-Middle* technique simply uses the `AdversaryInTheMiddle` attack step as a means of reaching `Eavesdrop`. Triggering the `Eavesdrop` attack step allows the attacker to `Read` any `Data` in transit over the `Network`. This scenario is presented in Fig. 19, where an attacker that has `FullAccess` on an uninspected `Network` can perform `Eavesdrop` on that network.

4.4. Impact

The *Impact* tactic disrupts the availability and integrity of software services and data. It consists of thirteen techniques, most of which were coalesced in one of the two *categories* introduced, *Application Denial of Service* and *Data Disruptions*, as shown in Fig. 20.

4.4.1. Application denial of service

This category groups techniques that deal with the attacker's ability to disrupt particular `Applications` and make them unavailable. There are several ways of denying (reaching the `Deny` attack step on) an `Application` in coreLang. First, if the attacker can get

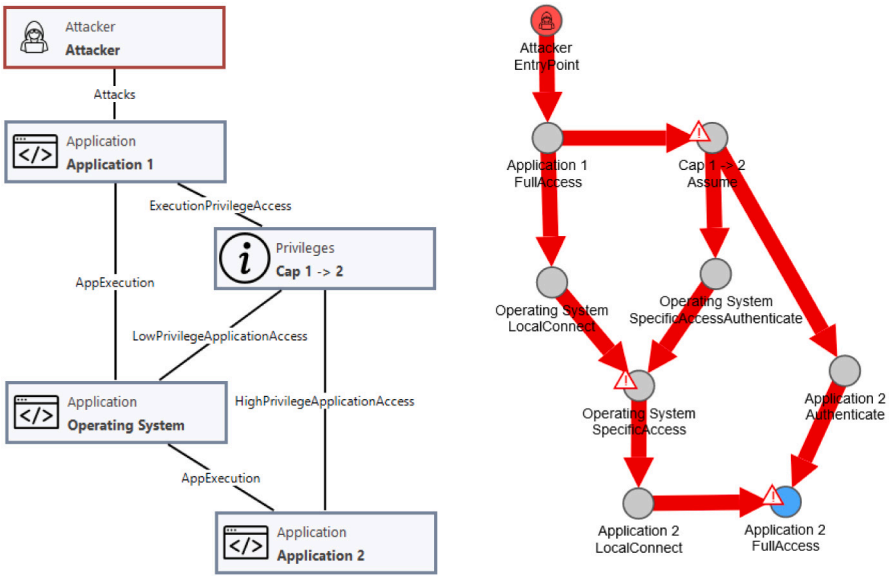


Fig. 14. *Inter-Process Communication* with IPC capabilities in coreLang.

`FullAccess`.[15] they are also assumed to be able to `Deny` the software representing the `Application`, by simply shutting it down, as described in the *Service Stop* and *System Shutdown/Reboot* techniques.

Second, denial of service via networking components, as mentioned in *Network Denial of Service* technique, is another common method of limiting the availability of services. It is represented in coreLang by denying an `Application` if all of its associated networking connections[16] suffer a `Deny`. Fig. 21 shows how this can be represented in the language. In the case of a partial disruption, where some of the networking components connected to the `Application` are still available, the `Deny` step on it will not be triggered because it is assumed capable of performing some of its duties to some degree. However, the extent of the attack can still be observed on the networking assets (`ConnectionRules` and `Networks`) impacted by it. Additionally, because coreLang does not model metrics such as available bandwidth and traffic volume, some aspects of network denial of service are more abstract. Since availability is binary, an asset is either denied or not.

Third, the attacker might be able to exploit a `SoftwareVulnerability` that impacts availability[17] of the `Application`. This is how coreLang represents features or flaws of software components that can lead to flooding or crashes, which are included in the *Endpoint Denial of Service* technique. Fig. 22 shows how this can be represented in the language.

Last, The *Account Access Removal* technique tackles the ability of an attacker to limit the availability of an `Application` by removing their accounts which are represented by `Identities` in coreLang. The example model in Fig. 23 illustrates how an attacker can lock out users by deleting their login `Credentials`. The `Delete` on the `Data` propagates to the `Credentials` contained, which in turn leads to a `Lockout` on the `Identity`. If the `Identity` had multiple `Credentials` that it could use to authenticate, all of them would need to be denied for the `Identity` to be locked out. Because the `Identity` in the example model has `ExecutionPrivilegesAccess` on the `Application` locking out the `Identity` leads to a `Deny` on the `Application` since there are no other execution permissions available[18]

4.4.2. Data disruptions

The other major disruption covered by the *Impact* tactic is targeting data. These translate to `Deny`, `Delete`, or `Modify` on the `Data` asset. Conceptually it is also analogous to the *Data Hosted on Applications* category, discussed in Section 4.3.1 in the *Collection* tactic, with the main difference between the two is that for the *Impact* tactic the attacker was seeking to deny, delete, and write data, while for the *Collection* tactic the goal is to read them. For example, the *Data Destruction* and *Disk Wipe* techniques are represented in coreLang by reaching `Delete`, while *Data Manipulation* and *Defacement*.[19] are represented via `Write`. Even though encryption is covered by coreLang, by having `Credentials` encrypt `Data`, it is only for defensive purposes in preventing the attacker from being able to access the data if they are not capable of decrypting them. Therefore, the *Data Encrypted for Impact* technique is represented in coreLang by having the attacker reach the `Deny` attack step on `Data`, since the encryption of the data by the attacker is one of the multiple ways that the data can be made unavailable to legitimate users.

4.4.3. Inhibit system recovery

The *Inhibit System Recovery* technique does not describe a freestanding procedure but rather a way to exacerbate an existing disruption to `Data` assets. It addresses steps taken by the attacker that prevent system recovery tools from restoring the data that the operation seeks to affect. To represent the backup volumes used by the system recovery tools, the `Replica` association between `Data` and `Information` can be used. Multiple `Data` instances can be assigned as replicas of the same `Information` instance; this means that an attacker is only able to reach `Deny`, `Delete`, or `Write` on that `Information` asset only

[15] The *Application Code Execution* category, Section 4.2.1, explains how the attacker would be able to get `FullAccess` on an `Application`.

[16] Typically these networking connections are represented by `ConnectionRules`, but `Networks` can also be directly associated to `Applications`.

[17] For a `SoftwareVulnerability` to restrict the availability of the associated `Application` it must not have the `AvailabilityImpactLimitations` defense enabled.

[18] If multiple `Identity`, `Group`, and/or `Privileges` which grant `ExecutionPrivilegesAccess` on the `Application` exist all of them need to suffer a `lockout` for the `Application` to be denied, otherwise it is assumed that users would just use one of the permissions still available if only some are locked out.

[19] *Defacement* is equivalent to *Data Manipulation* when representing it in coreLang since it simply defines a specific location and type of `Data`.

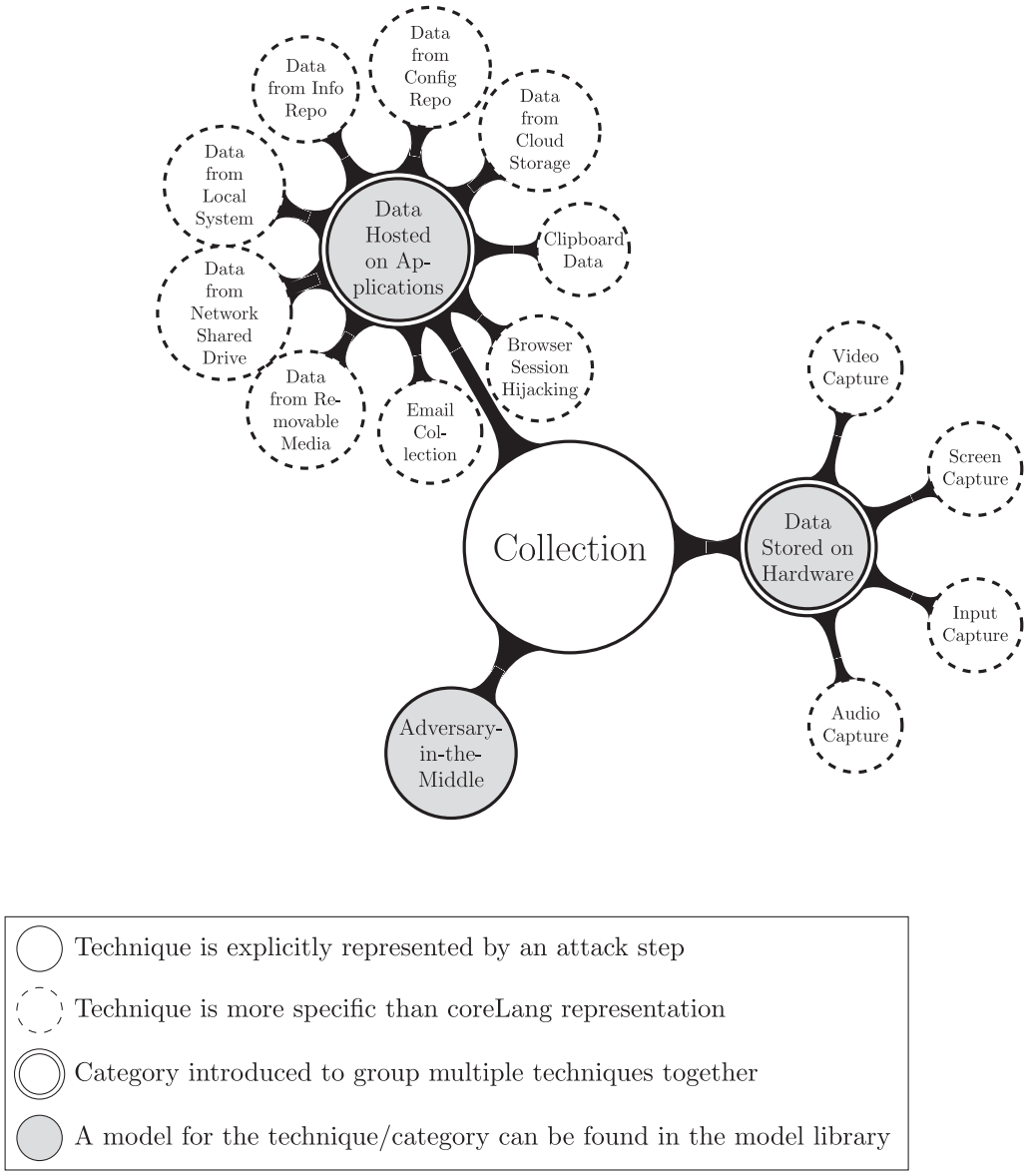


Fig. 15. Collection mindmap.

if the attacker can reach the corresponding attack step on all of the `Data` replicas.

4.4.4. Firmware corruption

The *Firmware Corruption* technique has the attacker targeting the firmware on devices to disable them. Depending on the specific aspects that scenario entails, firmware can be represented as either an `Application` or `Hardware` asset. The *Firmware Corruption* technique specifically mentions disrupting hardware devices; therefore, we have chosen to model it as `Hardware` in this instance. An attacker that gains `FullAccess` on an `Application` is also able to use any `HardwareVulnerabilities` associated with the `Hardware` asset hosting the `Application`. Similarly to how we used the `Software-Vulnerability` that reduced availability in the *Application Denial of Service*, we represented the lack of firmware tampering verification through a `HardwareVulnerability` and associated it with the `Hardware` on which the `Application` representing the operating system is running. After that vulnerability is exploited, and if the `AvailabilityImpactLimitations` defense is not enabled, `Deny` attack step on the `Hardware` will be achieved, which will then propagate to the `Application` running on it.

4.5. Credential access

This tactic deals with techniques to obtain credentials to perform further attacks. As described in Section 2.2.4, `Credentials` in coreLang are usually contained in `Data`, which means that many of the techniques in this tactic detail methods of accessing the `Data` containing `Credentials`. The *Credential Access* tactic consists of seventeen techniques. Thirteen of these techniques were grouped into two categories: *Extract Credentials* and *Intercept*. At the same time, the four sub-techniques of the *Brute Force* technique were dealt with individually. A visual representation of this can be seen in Fig. 24.

4.5.1. Extract credentials

Several of the techniques in the *Credential Access* tactic can be grouped under this broad concept of *Extracting Credentials*. The general idea is that the attacker can access an `Application` that

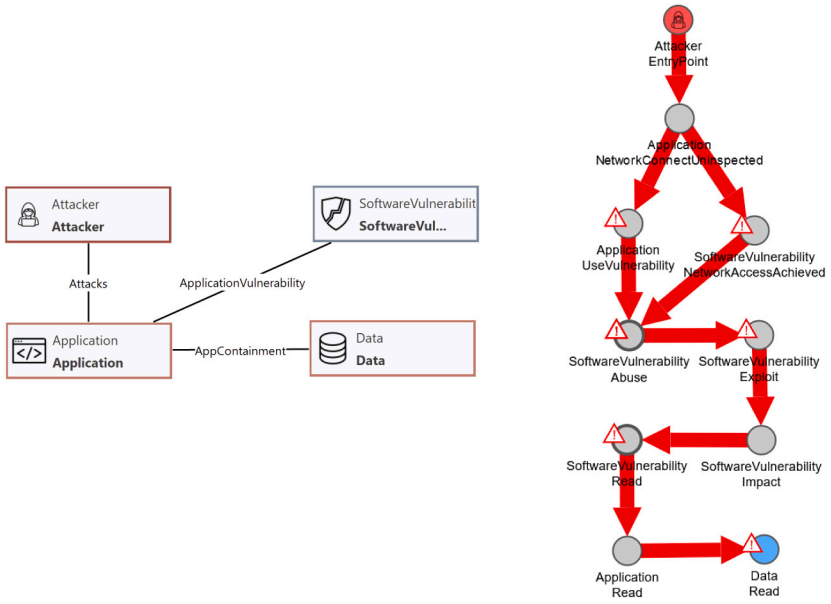


Fig. 16. Accessing *Data Hosted on Applications* using a `SoftwareVulnerability` in coreLang.

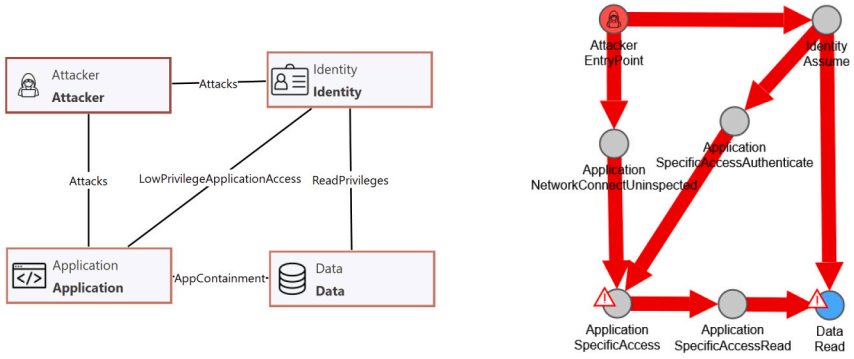


Fig. 17. Accessing *Data Hosted on Applications* using an assumed `Identity` in coreLang.

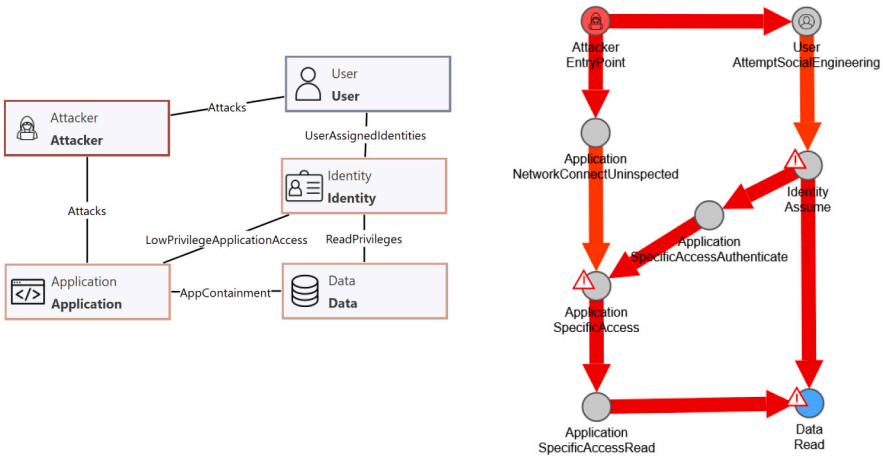


Fig. 18. Accessing *Data Hosted on Applications* using `socialEngineering` in coreLang.

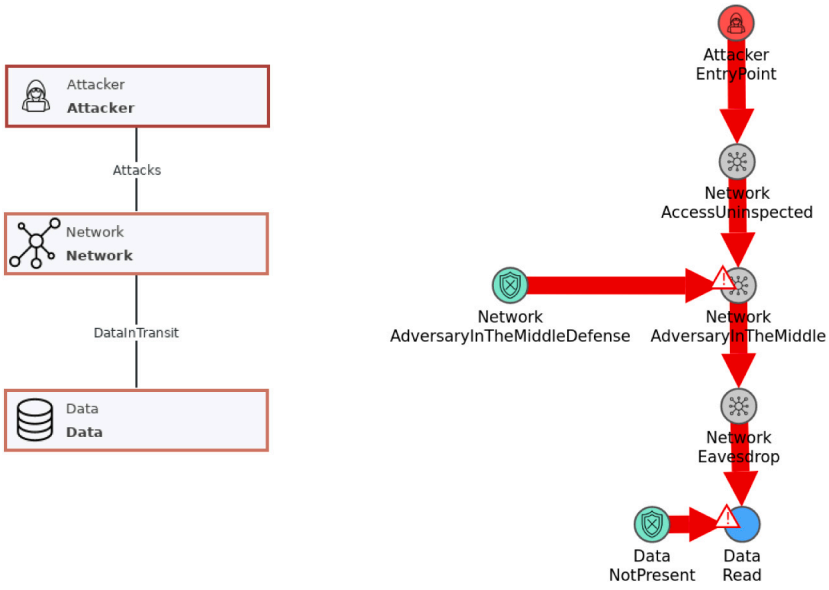


Fig. 19. *Adversary-in-the-Middle* in coreLang.

stores/contains the desired `Credentials`. Typically, an attacker gains access to `Credentials` by accessing the `Data` that contains them. Therefore, these techniques can be modeled in the same ways as with the *Data Hosted on Applications* category under the *Collection* tactic, Section 4.3.1. The only additions to the model in Figs. 16 and 17, would be introducing `Credentials` contained inside the `Data` using the `InfoContainment` association.

Credentials from Password Stores, *Exploitation for Credential Access*, and *Unsecured Credentials*.[20] are in line with the general description above. *Input Capture*, *OS Credentials Dumping*, *Steal Application Access Token*, and *Steal or Forge Kerberos Tickets* all define specific types of `Credentials` to be stolen or particular `Applications` hosting those `Credentials`

4.5.2. Intercept

One of the techniques in this category, *Adversary-in-the-Middle*, was already covered in the *Collection* tactic, Section 4.3, and similarly to the *Extract Credentials* category, the description in the *Adversary-in-the-Middle* under Section 4.3.3 is still applicable. The aspects of the technique on *Credential Access* involve simply adding `Credentials` that are contained in the `Data`. *Network Sniffing* is implemented by having the attacker reach the `Eavesdrop` attack step on the `Network` asset over which the `Data` are transiting, this fits the same logic as well. *Multi-Factor Authentication Interception*.[21] can be implemented by gaining access to the additional factor over the `Network` rather than the main `Credentials`.

4.5.3. Brute force

The *Brute Force* is one of the rare instances where coreLang is more detailed in its implementation than the ATT&CK technique, so we validate against each of four of its sub-techniques instead. *Password Spraying* and *Credential Stuffing* are implemented via the `NotDisclosed` defense on the `Credentials` asset, which, if disabled, results in the direct compromise of the asset without necessitating attacker activity.

Password Guessing is implemented using the `NotGuessable` defense on the `Credentials` asset to represent the fact that `Credentials` can be guessed.

Password Cracking is implemented via the `HashedCredentials` association between two `Credentials` assets that represent the actual credentials and their hashes, respectively. The credentials can be guessed if the attacker gets access to the `Credentials` representing the hashes.

4.6. Privilege escalation

Because many of the techniques in this tactic (see Fig. 25) deal with abusing access control privileges and manipulating execution contexts, they resemble similar ones that were discussed in *Initial Access*, *Credential Access*, and *Execution* tactics. As such, we will only briefly describe these techniques and emphasize how they distinguish themselves from their previously covered counterparts.

The *Valid Accounts* technique was already addressed in the *Initial Access* tactic when discussing the *Exploit Account* category, Section 4.1.2.

4.6.1. Access token manipulation

The *Access Token Manipulation* technique involves creating, stealing, forging, and impersonating existing tokens. This is comparable to the *Steal Application Access Token*, *Steal or Forge Kerberos Tickets*, and *Steal Web Session Cookie* techniques found under the *Extract Credentials* category, introduced on the *Credentials Access* tactic, which is described under Section 4.5.1. The distinguishing characteristic of the tokens mentioned in the *Access Token Manipulation* technique in this tactic is that they are typically meant to be used on the same system and provide a means of leveraging existing privileges to obtain elevated ones.

4.6.2. Exploitation for privilege escalation

The *Exploitation for Privilege Escalation* technique is very similar to the *Application Code Execution* category found in the *Execution* tactic, under Section 4.2.1. Both deal with software vulnerabilities that enable attackers to gain elevated privileges on a particular application. The only difference is that the *Exploitation for Privilege Escalation* suggests that the `SoftwareVulnerabilities` used by the attacker requires lower privileges to abuse the privilege escalation mechanisms and, therefore, should have the `LowPrivilegesRequired` defense enabled to reflect that.

[20] *Unsecured Credentials* also suggests that part of the problem is the lack of encryption. How encryption is represented in coreLang can be found in Section 4.8 *Mitigations*, when discussing the *Encrypt Sensitive Information* mitigation.

[21] How multi-factor authentication is implemented in the language can be found in the *Mitigations* Section 4.8.

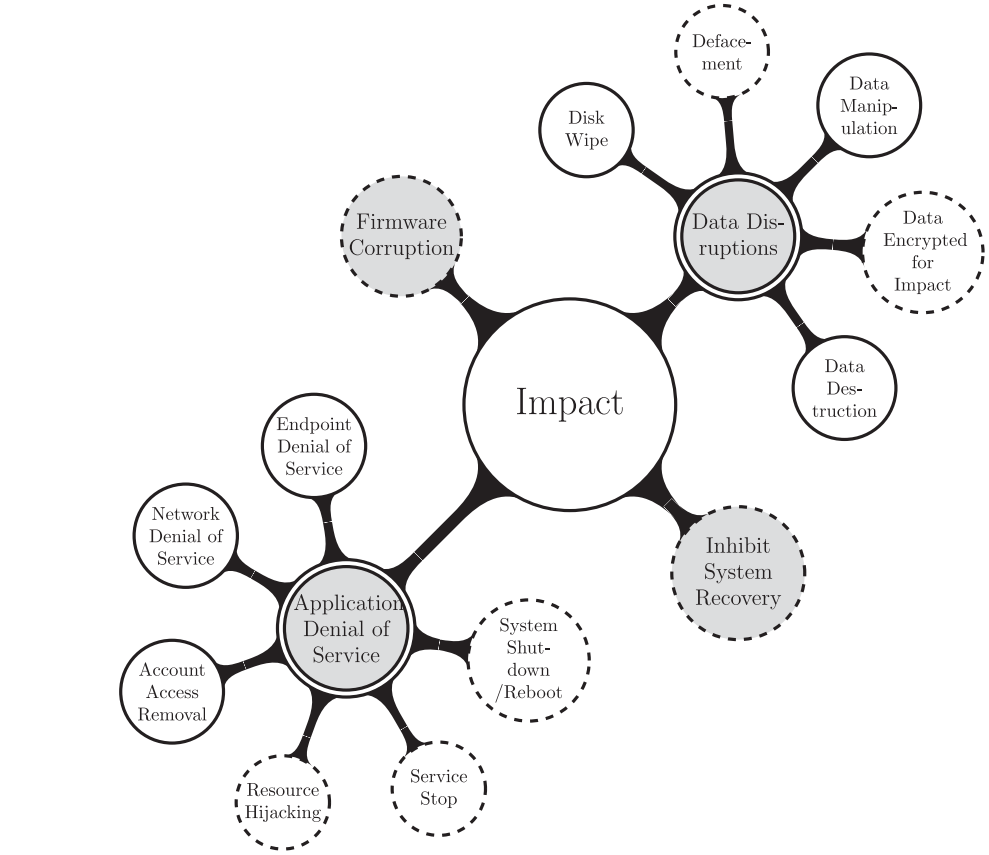


Fig. 20. Impact mindmap.

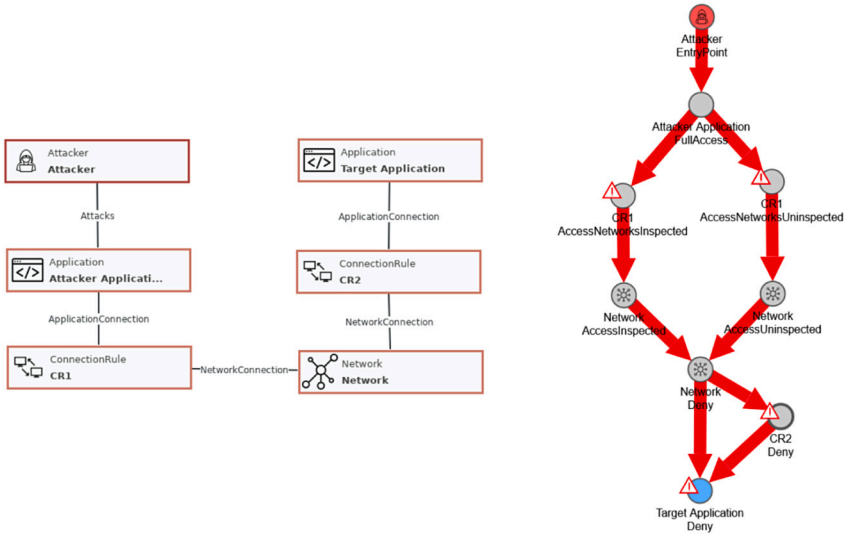


Fig. 21. *Application Denial of Service* through `Networking` components in coreLang.

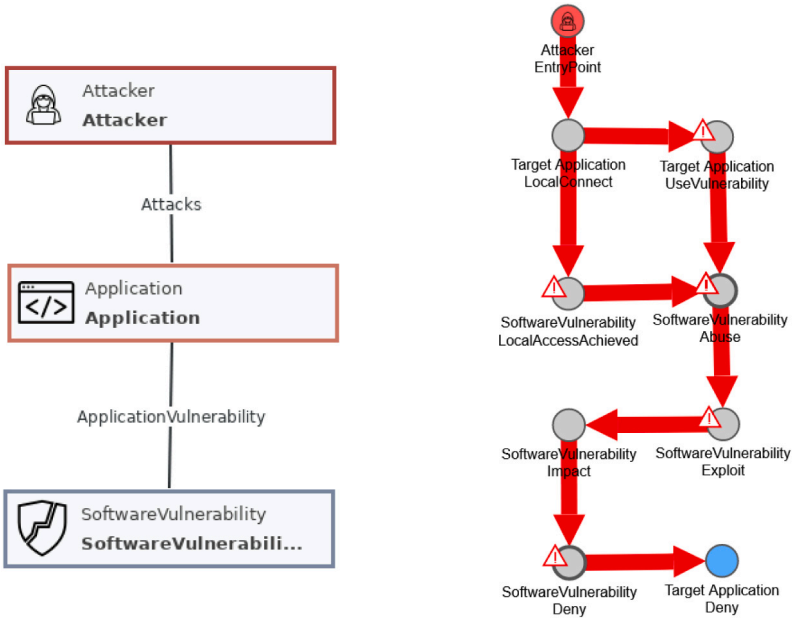


Fig. 22. *Application Denial of Service* through a `SoftwareVulnerability` in coreLang.

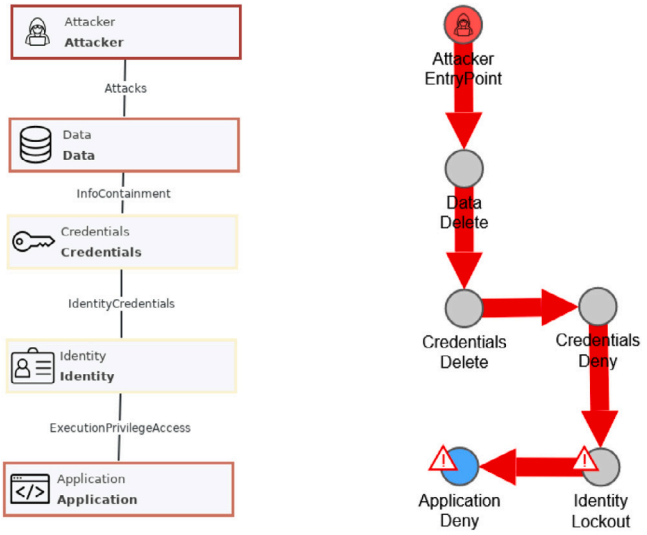


Fig. 23. *Application Denial of Service* through *Account Access Removal* in coreLang.

4.6.3. Domain policy modification

This technique encompasses many attacks under its umbrella that involve the attacker modifying domain policy entries to attain access control privileges. Domain policy entries are generally represented in coreLang using `IAMObject` assets. All three assets that extend the abstract `IAMObject` class, namely `Identity`, `Group`, and `Privileges`, can manage other `IAMObject` assets,[22] this is done via the `AccountManagement` association. Because many domain policy services allow exceptions to be specified regarding which parts of a hierarchy a particular configuration applies to, the domain policy hierarchy structure must be explicitly modeled in coreLang as manage associations between the `IAMObject` representing the permissions and the ones depicting the entries to which it applies. The `IAMObject` class extends `Information`, and the modeler can, therefore, specify `Data` assets that contain the domain policy configuration, which an attacker could, in turn, modify to gain access to permissions defined. Specifically for the *Group Policy Modification* sub-technique, a `Group` can represent a set of privileges assigned to the domain policy entry associated with the group in the domain that the attacker could target.

4.6.4. Escalation mechanisms

This large category subsumes all techniques relying on privilege escalation mechanisms on a particular system to attain elevated privileges. There are two ways to interpret the attacker's abuse of the privilege escalation features in coreLang. First, they can be seen as built-in vulnerabilities of the tools or operating system features, which would be depicted similarly to the *Exploitation for Privilege Escalation* technique mentioned above. Second, and more often the case, the modeler aims to represent a situation where the attacker is able, from within their current execution context, to change a configuration that allows them to execute code in an elevated execution context. An example of this can be seen in Fig. 26. The attacker can leverage the privileges available in the initial execution context to write/modify the configuration file of the `Target App` and gain access to its execution context. This is achieved through the `Dependence` association in coreLang, which is used to specify `Data` (or `Information`) that act as a form of dependence (i.e., such as a configuration file) to an `Application`. More specifically, `Dependence` should be used to represent various situations where the operations of the `Application` are impacted by modifying/denying the `Data/Information` it depends upon. Examples of this type of behavior are configurations, plugins, libraries, or other input data that is critical to the purpose of the software component represented by the `Application`. This association should only be used for circumstances where the workflow of the `Application` is not altered, but the `Data/Information` are modified within the regular operating process.

4.6.5. Escape to host

Sandboxing in coreLang is implemented by having `Applications` host other `Applications`. If an attacker can compromise one of the sandboxed containers, the attacker's control is limited to it. However, from within the container, they can attempt to use privileges

[22] Other access control levels generally found in domain policy services apart from manage/write (e.g. read, search) are not particularly relevant for coreLang since the language does not cover reconnaissance and assumes that the attacker can get all the required information regarding the domain policy structure.

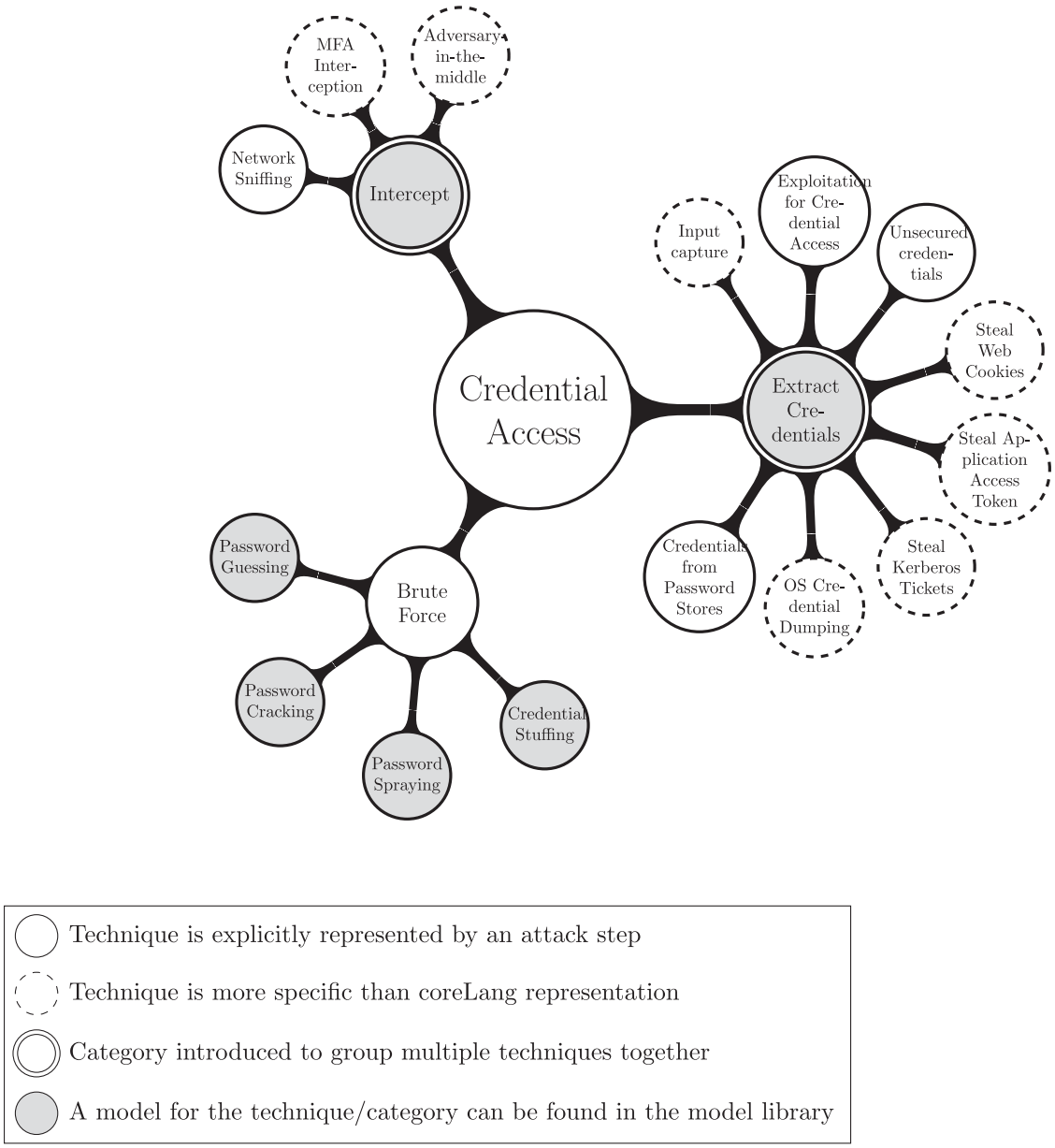


Fig. 24. Credential access mindmap.

on the host they may have otherwise attained or abuse vulnerabilities on the host to extend their reach to the host environment (this is achieved via the `BypassContainerization` attack step of the `Application` asset). Therefore, although broad, this technique is easily implementable by either connecting the relevant `Identities` or `SoftwareVulnerabilities` to the `Application` representing the host. After this, the same mechanisms as *Application Code Execution*, which are described under Section 4.2.1, are used.

4.7. Lateral movement

Lateral movement refers to the actions taken by an attacker to expand their reach within an already compromised network. coreLang does not distinguish between internal or external networks, and they are simply different asset constellations that the modeler can employ to depict the desired behavior. As such, some of the techniques subsumed in this tactic will be very similar to their counterparts, targeting either general or external components in other tactics. As shown in Fig. 27, one category, *Compromise Services*, was introduced.

4.7.1. Compromise services

The four techniques grouped under this category, *Exploitation of Remote Services*, *Remote Service Session Hijacking*, *Remote Services*, and *Software Deployment Tools*, all deal with the attacker getting access to particular services found on an internal network and extending their reach from there. They all use concepts presented in other tactics but are specifically applied to expanding access to other systems within a network from a compromised machine. *Exploitation of Remote Services* is similar to the *Application Code Execution* category in the *Execution* tactic, Section 4.2.1. *Remote Services* employs the same mechanisms as shown in the *Valid Accounts* technique in the *Initial Access* tactic, Section 4.1.2, but applies them to multiple systems on the same network where the `Credentials` grant access. *Software Deployment Tools* was included

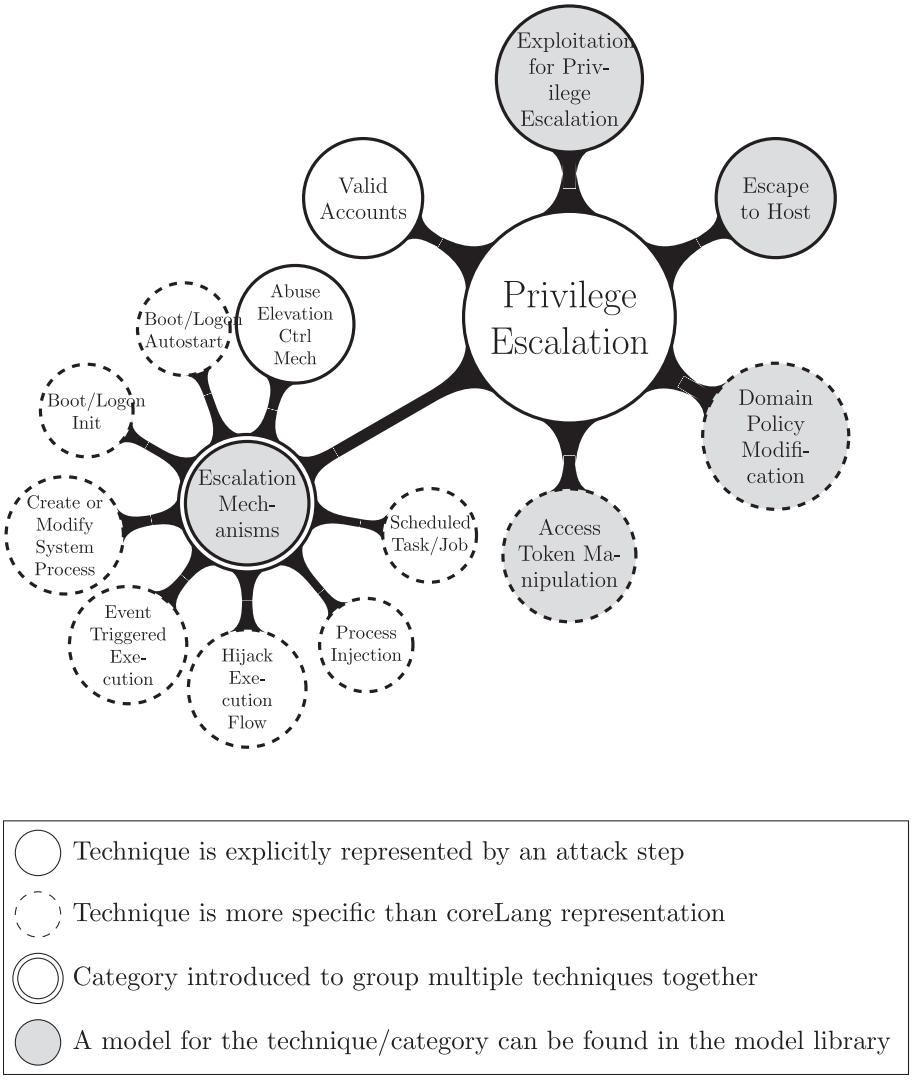


Fig. 25. Privilege escalation mindmap.

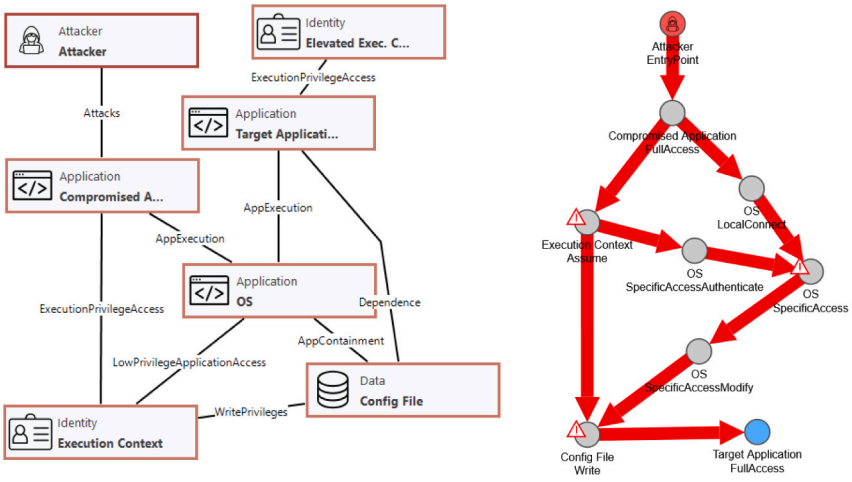


Fig. 26. Complex scenario for *Escalation Mechanisms* category in coreLang.

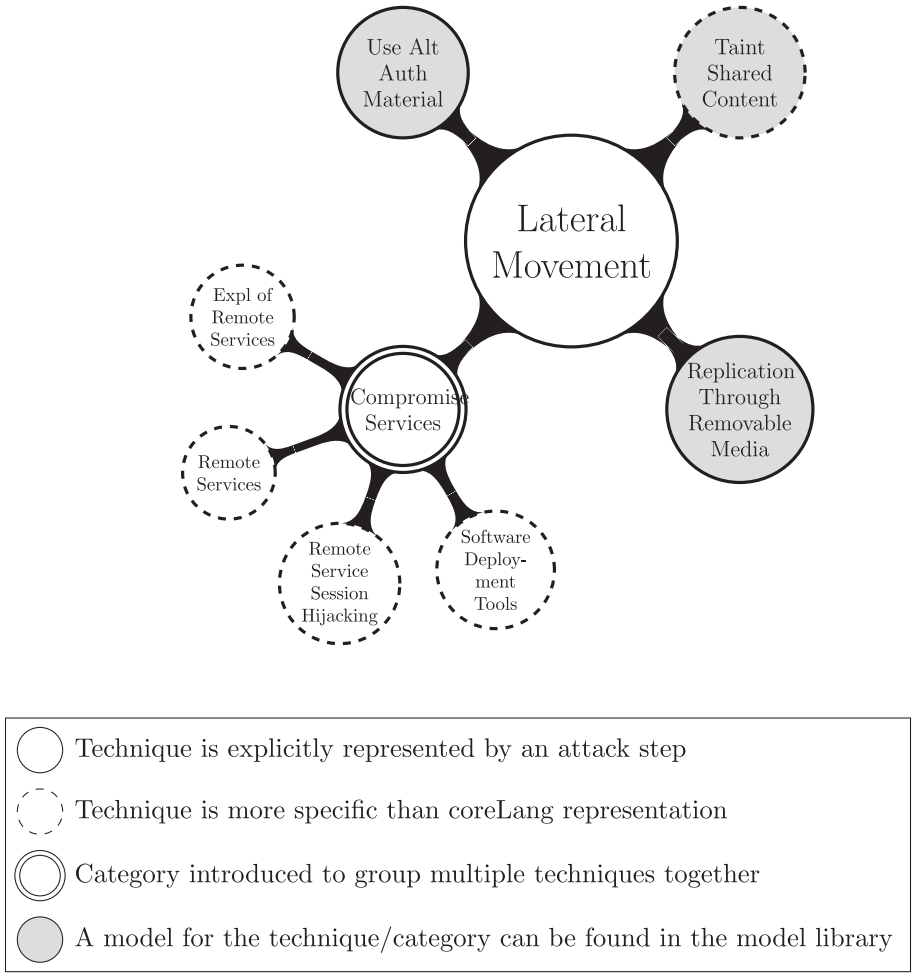


Fig. 27. Lateral movement mindmap.

in the *Application Context Hijacking* category in *Execution* tactic, Section 4.2.2. That discussion also covers lateral movement aspects, which can also be used to represent *Remote Service Session Hijacking*.

4.7.2. Replication through removable media

The initial intrusion aspects of this technique have already been discussed in Section 4.1 Initial Access. The facets that deal with *Lateral Movement* revolve around spreading malware from an already compromised system to others via removable drives. If the attacker can gain `FullAccess` on an `Application`, they can attempt to spread malware via removable media. If a `User` that utilizes removable drives[23] has access to the `Hardware` system on which the compromised `Application` is running they will spread the malware to any other `Hardware` system that they have access to. As seen in Fig. 28, the representation is very similar to the one presented in the *Initial Access* tactic, Section 4.1, in Fig. 8. The additional components pertinent to the *Lateral Movement* tactic introduced are the Initial Application and Initial Hardware from which the malware spreads.

4.7.3. Taint shared content

This technique involves having the attacker modify data stored on a shared network location by inserting a malicious payload that infects the systems that access the content. To replicate this behavior, the `Dependence` association between `Data` and `Applications` can be used to represent the fact that, if the attacker alters the shared storage, the software applications that use it will also be compromised. Fig. 29 shows an example model of this technique. For simplicity, the `Attacker` was given `FullAccess` on the Network Storage `Application` since the scenario mentioned by the technique presupposes that the attacker can manipulate the network storage data. Being able to `Write` to the Shared Content `Data` asset means that the attacker can compromise both Application 1 and Application 2, which are two `Applications` that are reliant on the Shared Content. The `Dependence` association can be used between `Applications` and either `Data` or `Information` depending on the scenario in the case. If the attacker can write the `Data` or `Information`, they will get `FullAccess` on the associated `Applications`.

4.7.4. Use alternate authentication material

The four sub-techniques that comprise this technique use application access tokens, hashes, tickets, and web session cookies to authenticate instead of the primary credentials initially used to generate them. As highlighted when discussing the *Crendetial Access* tactic, Section 4.5, coreLang allows for relatively complex arrangements of authentication mechanisms to be represented. `Credentials` can be used as required factors for other `Credentials`, usually to represent multi-factor authentication, or as hashes of other `Credentials`, specifically to allow an attacker to gain permissions based on the hashes or to facilitate *Password Cracking* as seen in the *Brute Force* technique, Section 4.5.3. For situations where the alternate authentication material is session or context-specific, the relevant `Credentials` can be stored via `Data` in the `Application` depicting the particular

[23] The `User` does not have the `NoRemovableMediaUsage` defense enabled.

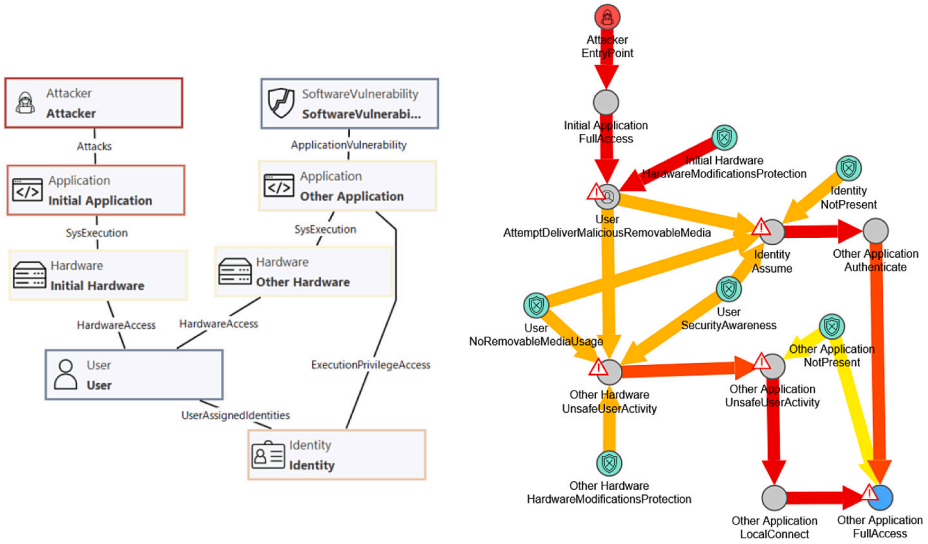


Fig. 28. *Replication Through Removable Media* technique in coreLang.

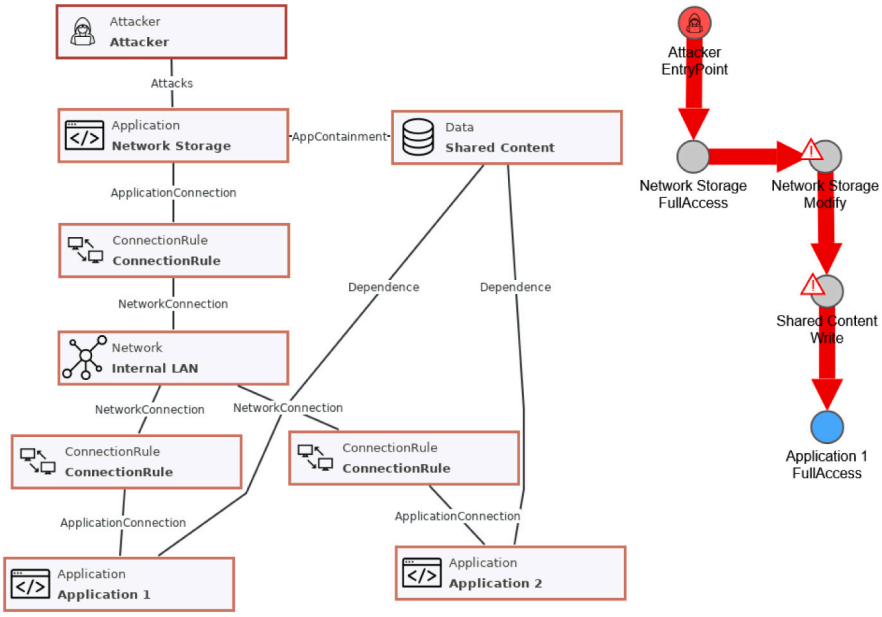


Fig. 29. *Taint Shared Content* technique in coreLang.

instance. Since any `Credentials` asset can be used to assume an `Identity` in coreLang, any of the `Credentials` in the authentication chain, whether they be the primary, additional factors, hashes, or generated tokens, can be used by the modeler to grant the relevant permissions. The example model in Fig. 30 shows how a `Credentials` `Hash` utilized in the authentication process can be used by an attacker to authenticate on the `Application` without needing the original `Credentials`, and by only assuming a network connection towards the target `Application`.

4.8. Mitigations

So far, we have addressed how various ATT&CK techniques are represented in coreLang. This section will explain how defensive measures can be implemented in the language, referred to as *mitigations* by ATT&CK. There are two distinct ways of achieving this. The first is using defenses on assets, which prevent specific attack steps from being reached, such as the `PayloadInspection` defense which prevents the `AccessNetworksUninspected`, `ConnectToApplicationsUninspected`, and `ReverseReach` attack steps on the `ConnectionRule` asset. The second one is with defensive assets, where an association between two assets restricts the attacker's ability to perform specific actions, such as the `Credentials` asset that can be used to sign or encrypt `Data` assets. There are several structural defenses in coreLang which combine aspects of both. Structural defenses make certain assets behave as if they were not present in the model at all, thus removing all the attack vectors going through them. These defenses are typically called `NotPresent`, for the `ConnectionRule` asset the structural defense is called `Restricted` because it better describes the network traffic restriction.

Because the same ATT&CK mitigation usually applies to multiple techniques, frequently under different tactics, these mitigations outline various aspects in their descriptions. Many mitigations describe very specific actions that can be taken to prevent certain techniques. For example, *Encrypt Sensitive Information* for the *Data from Configuration Repository* technique advises "Configure SNMPv3 to use the highest

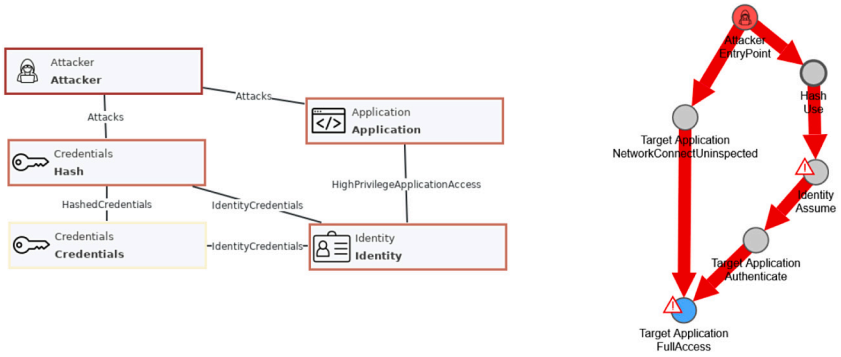


Fig. 30. *Use Alternate Authentication Material* technique utilizing a `Credentials` hash in coreLang.

level of security (authPriv) available".. These implementation specific descriptions were not taken into consideration because we focused on the general concept each mitigation covers and described how it could be represented in coreLang models. Below, we explain some of the more relevant mitigations for coreLang, but a full list can be found in the mitigations section[24] of the online language documentation page (Buhaiu, 2022).

Antivirus/Antimalware

Antivirus or antimalware tools are explicitly represented in the language by the `IDPS` asset that is used to safeguard `Applications` from `SoftwareVulnerabilities` and `UnsafeUserActivity` that the attacker induced through `SocialEngineering`. The `IDPS` asset has an `Effectiveness` defense, representing how well it can perform its protective duties. The higher the `Effectiveness` is the better it will be able to prevent the attacker from reaching the relevant attack steps[25] on the associated `Applications`.

Code signing

The `SigningCredentials` association between `Credentials` and `Data` can be used to represent the signing of any `Data`. The signed `Data` cannot be overwritten by the attacker unless they have compromised the signing `Credentials`.

Often, this signing mitigation is suggested for release binaries or source code. For these situations, the `Data` can then be specified as containing a `SoftwareProduct` designated as the software package used by one or more `Applications`.

Data backup

The `Replica` association between `Information` and `Data` was specifically introduced during the validation process to implement this mitigation. An attacker that can `Read` a replica is granted `Read` on all other replicas since they contain the same `Information`. However, for the `Information` to suffer a `Write`, `Delete`, or `Deny`, all of its replicas need to have been impacted by `Write`, `Delete`, or `Deny`, respectively. The language assumes that any of the replicas can be used to recover the `Information`. This means it does not cover more intricate scenarios, such as gradual poisoning of the backups or exploiting voting system mechanisms.

Encrypt sensitive information

This can be represented in coreLang by using `Credentials` to encrypt the relevant `Data`. The encrypted `Data` cannot suffer `Read`, `Write`, `Delete` by the attacker unless they were able to compromise the encryption `Credentials`. However, the encrypted `Data` can still be affected by `Deny`.

Filter network traffic

Protocol-based traffic filtering is implemented in coreLang by using the `Restricted` defense on `ConnectionRule` or by changing the model structure and removing `ConnectionRules`. The `Restricted` structural defense stops all traffic through the `ConnectionRule` asset. This makes the `ConnectionRule` behave as if it was not present in the model, which aligns with the structural approach of simply removing the asset. However, `Restricted` is assumed to be imperfect and that the attacker may find ways to bypass the filtering with substantial effort, as will be shown in the *Defense Evasion* conversation in Section 4.9.

Multi-factor authentication

The `ConditionalAuthentication` association between `Credentials` represents multi-factor authentication in coreLang. For the attacker to compromise primary credentials all of the additional factors, the `Credentials` that are defined as additional factors using the `ConditionalAuthentication` association, need to be compromised as well. This association can be nested, and multiple required factors can be specified for the same `Credentials`.

Furthermore, three defenses on the `Credentials` asset exist that allow the modeler to define its characteristics, `NotGuessable`, `Unique`, and `NotPhishable`. These allow `Credentials` to represent a variety of authentication mechanisms, from biometrics to one-time passwords.

Network intrusion prevention

Payload-based filtering in coreLang is implemented via the `payloadInspection` defense on the `ConnectionRule` asset. This defense prevents the attacker from being able to exploit `SoftwareVulnerabilities` or induce `UnsafeUserActivity` on the `Applications` they reach via inspected `ConnectionRules`, as the payload inspection is expected to detect the anomalous patterns and filter the traffic out. However the adversary would still be able to utilize legitimate `Credentials` to `Authenticate`, since these would not stand out as atypical.

The `AdversaryInTheMiddleDefense` defense on the `Network` asset can also be used to protect `Data` that transit the `Network` from *Adversary-in-the-Middle* attacks, that are originating either form the network layer (like ARP spoofing attacks) or by manipulating the higher network layers (like DNS poisoning attacks).

Restrict file and directory permissions

`[Read|Write|Delete]Privileges` can be defined for IAM assets (`Identity`, `Group`, or `Privileges`) to designate their permissions on `Data`, which is how files and directories are represented in coreLang. This mitigation suggests structural changes to the models by removing extraneous privileges. The `NotPresent` structural defense on the IAM assets can be used to speculate about the attack vectors that removing these permissions would thwart.

[24] https://mal-lang.org/coreLang-documentation/mitre/mitigations.html.

[25] The two attack steps an `IDPS` specifically prevents are `UseVulnerability` and `UnsafeUserActivity`.

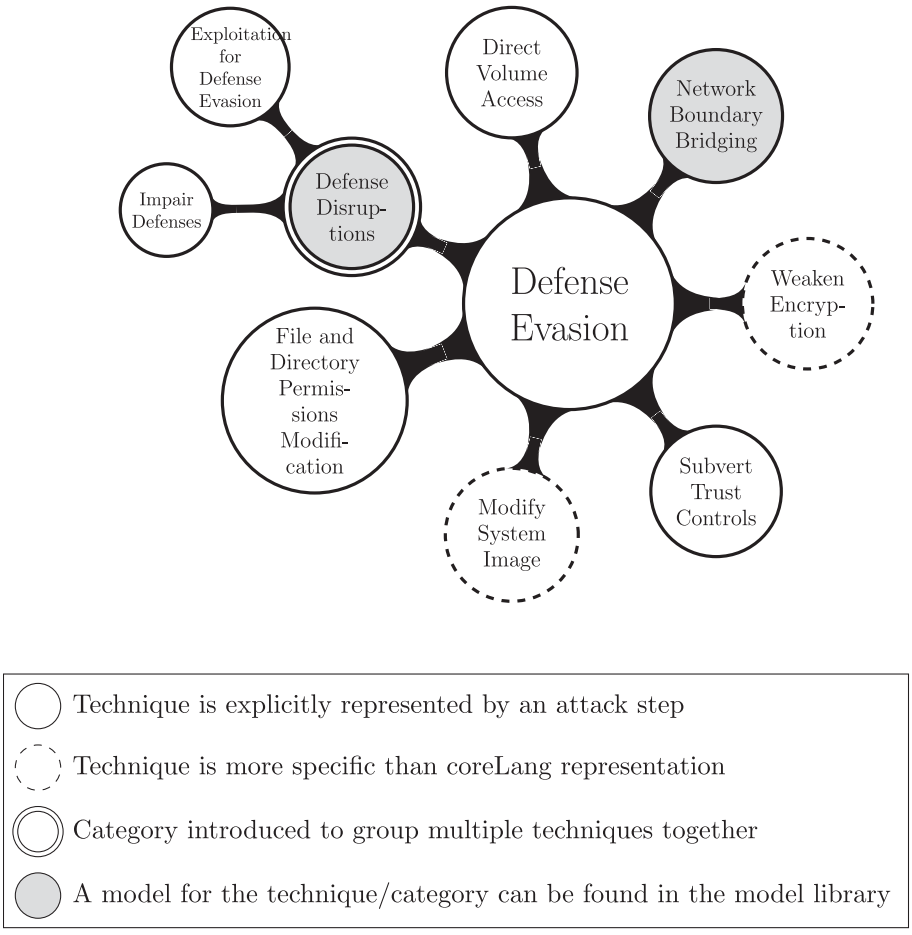


Fig. 31. Defense Evasion mindmap.

User training

The principal way to represent this technique in coreLang is the `SecurityAwareness` defense on the `User`. This defense prevents a variety of disruptions enabled by `SocialEngineering`, such as phishing the `User` for `Credentials` or tricking them into performing unsafe user actions on `Applications`.

4.9. Defense evasion

This tactic deals with an adversary's actions to avoid detection and disable defenses. As such, it intersects with many other tactics and contains a very large number of techniques (see Fig. 31). To discuss these techniques in a manageable fashion, we have decided to separate them into three distinct groupings. First, we will look at the ones covered by the language with the help of a mindmap. Second, the techniques already discussed in other tactics will be presented in a table. Lastly, we will provide a table containing all the techniques not covered and why they do not fit the language well.

4.9.1. Techniques covered

coreLang provides two ways of bypassing the mitigations described in Section 4.8. The first is conditional bypasses, where the actions of an attacker disrupt or impair the ability of defensive measures to perform their protective tasks. For example, since the `IDPS` asset extends the `Application` asset, an adversary could gain access to the `IDPS` and prevent it from safeguarding the `Applications` it is associated with. If the attacker reaches the `Deny` or `FullAccess` attack steps on an `IDPS`, it will no longer function as a defensive asset which means that the attacker can engage in the behavior described in the *Application Code Execution* category, Section 4.2.1. This behavior can be used to implement techniques such as *Exploitation for Defense Evasion*.[26] and *Impair Defenses*, as shown in Fig. 32.

Similarly, the `RoutingFirewall` asset extends `Application`. Suppose an adversary gains `FullAccess` to it. In that case, they can bypass any restrictions imposed by the existing `ConnectionRules`.[27] that the `RoutingFirewall` controls. These mechanisms can represent *Network Boundary Bridging*, as shown in Fig. 33

The other method of bypassing defensive measures in coreLang is through unconditional bypasses. These represent the fact that the defenses they bypass are imperfect. Therefore, they occur automatically when the attacker attempts to perform an action that a certain defense could prevent. For example, if the attacker is attempting to use a `ConnectionRule` to transmit their malicious payload and their ability to do so is restricted by the fact that `PayloadInspection` is enabled on that `ConnectionRule`. It is then assumed that the attacker will try to circumvent the payload filtering, represented by the `BypassPayloadInspection` attack step. All of the unconditional bypass attack steps, such as `BypassPayloadInspection`, `BypassAdversaryInTheMiddleDefense`, etc., enable alternative

[26] The *Exploitation for Defense Evasion* description is slightly more broad and ambiguous, which means that it could also just be interpreted as being functionally similar to the *Application Code Execution* category, discussed in Section 4.2.1.

[27] Because the attacker cannot insert assets as part of their actions, they cannot introduce `ConnectionRules` to extend their reach. Still, they can prevent any filtering, `PayloadInspection` and `Restricted`, on the existing ones.

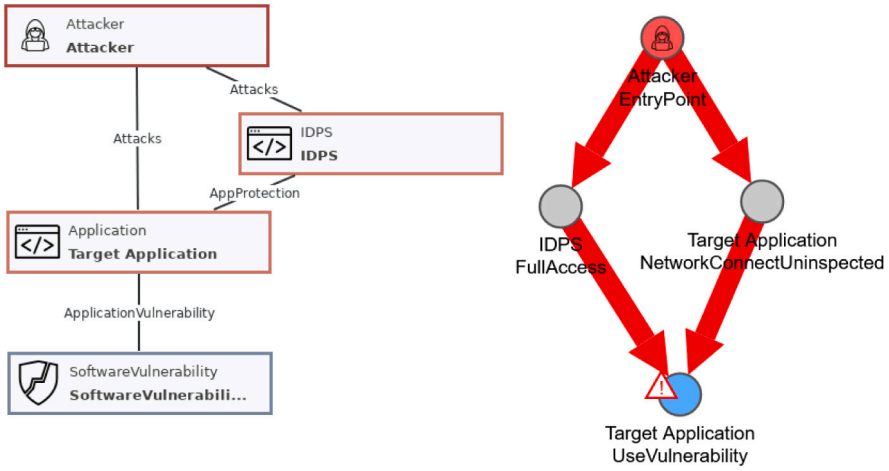


Fig. 32. *Impair Defenses* attack in coreLang.

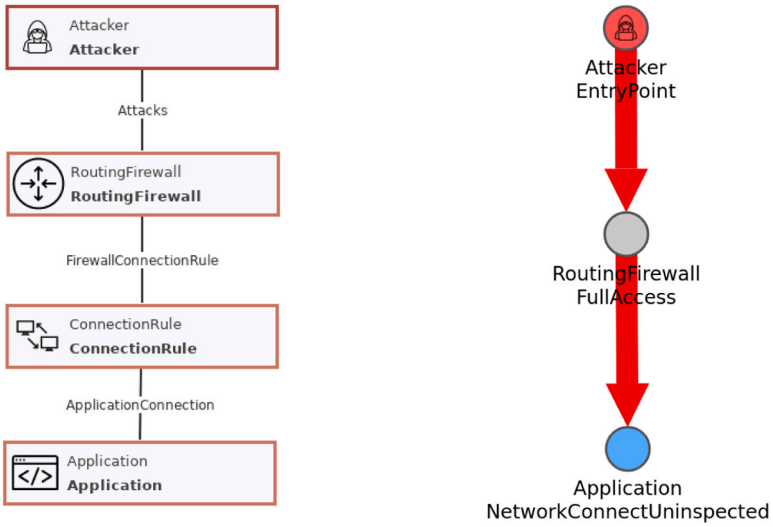


Fig. 33. *Network Boundary Bridging* attack in coreLang.

attack paths without any additional actions required from the attacker. Given significant additional effort, they represent the adversary's ability to evade these fallible defenses. This effort is represented via a time to compromise (TTC) distribution as explained in the Background Section 2.1. Additionally, a modeler can define their own TTC probability distributions that override the default ones in the language to better account for the specific bypasses they have in mind. It is possible for a defense to have both conditional and unconditional bypasses in the attack steps that they enable. An interesting example involving both bypasses is the *Weaken Encryption* technique. Manipulating or disabling the cryptographic functionalities of a network device can be implemented via conditional bypasses, `BypassEavesdropFromPhysicalAccess` and `BypassAdversaryInTheMiddleFromPhysicalAccess`, while reducing cipher strength aspects are subsumed in the unconditional bypasses, `BypassEavesdrop` and `BypassAdversaryInTheMiddle`.

File and Directory Permissions Modification can be implemented using the `AccountManagement` association described in Section 4.6.3 *Domain Policy Modifications*. For coreLang, *Direct Volume Access* is a specific case of the *Data Hosted on Applications* category, Section 4.3.1, where the `Data` are accessed from an `Application` representing the operating system. The *Modify System Image* technique can be implemented using the `Dependence` association mentioned in Section 4.6.4 by having `Data` represent the system image and an `Application` representing the operating system. As discussed in the *Code Signing* portion of the Mitigations Section 4.8, to `Write Data`, an attacker needs to be able to compromise any signing credentials if they are present. These mechanisms can represent the *Subvert Trust Controls* technique.

In Table 3 all the *Defense Evasion* techniques that were previously discussed as covered in other tactics are summarized, while in Table 4 the *Defense Evasion* techniques that are not covered are summarized.

4.9.2. Techniques discussed in other tactics

See Table 3.

4.9.3. Techniques not covered

See Table 4.

4.10. Exfiltration

Exfiltration is logically linked to the *Collection* tactic that was discussed in Section 4.3, since it deals with the attacker extracting the data after they have been collected. While coreLang does not clearly distinguish between collection and exfiltration of `Data` and `Information`, several aspects described in this tactic are beyond the scope of the language. As such, many techniques are too specific for coreLang's purposes and, therefore, are either implicitly modeled or not covered.

Table 3
Techniques included in the Defense Evasion tactic that are covered in other tactics.

Tactic	Technique	Subsection reference	Description/Notes
Privilege escalation	Abuse elevation control mechanism Access token manipulation Domain policy modification Hijack execution flow Process injection	4.6.4 4.6.1 4.6.3	All of these techniques bypass defenses by leveraging elevated privileges. This is implicitly modeled in coreLang by allowing `HighPrivileges` to perform more disruptive actions.
Execution	Deploy container	4.2.2	Covered by the *Application Context Hijacking* category.
Credential access	Modify authentication process	4.5.1	Covered by the *Extract Credentials* category.
Lateral movement	Use alternate authentication material	4.7.4	This technique involves secondary authentication material (*Use Alternate Authentication Material*, Section 4.7.4, gives an example of how alternative authentication material can be represented in coreLang).
Initial access	Valid accounts	4.1.2	Covered by the *Exploit Accounts* category.

Table 4
Techniques included in the Defense Evasion tactic that are not covered by coreLang.

Reason	Technique	Description/Notes
Ensure persistence	BITS Jobs Pre-OS boot Traffic signaling	As described in Section 4.11, due to the static nature of MAL languages attack graphs, *Persistence*, is something that we do not account for in this validation.
Bypass specific defenses	Build image on host Indirect command execution Modify cloud compute infrastructure Plist file modification System binary proxy execution System script proxy execution Template injection Trusted developer utilities proxy execution XSL script processing	These techniques deal with bypassing particular defenses and are such are too implementation specific for coreLang. They are either implicitly assumed to be included in the more general attack steps(i.e. **FullAccess**) or otherwise not covered at all.
Obfuscate adversary artifacts and actions	Debugger evasion Deobfuscate/Decode files or Information Execution guardrails Hide artifacts Indicator removal Masquerading Modify registry Obfuscated files or Information Reflective code loading Rogue domain controller Rootkit Unused/Unsupported cloud regions Virtualization/Sandbox evasion	These techniques deal with hiding the presence of a malicious attacker by concealing actions or artifacts that would expose their presence. Since coreLang does not track attacker artifacts in any way these techniques not covered at all.

This resembles the discussion around some of the techniques in the *Collection* tactic where *Automated Collection* and *Data Staged* were implied, while *Archive Collected Data* was not covered.

The general concept of exfiltrating data is explicitly covered by the language through the `Extract` attack steps found on `Data` and `Information`. The `Extract` attack step has two requirements: collection and reachability. First, it requires a `Read`, equivalent to the attacker collecting the data or information. Second, a network communication path must exist from where the `Data` is found to the attacker. This reachability requirement describes the ability of the `Data` to be sent to the attacker rather than the capacity of the attacker to reach the `Data`. The situation where the attacker can collect the `Data` but lacks the means to retrieve it via network connections represents autonomous tools using the `Data` to perform their attacks without attacker input.

Automated Exfiltration, *Exfiltration Over C2 Channel*, *Exfiltration Over Other Network Medium*, and *Exfiltration Over Web Service* are all implementation-specific details that are just implied in coreLang attack steps. *Data Transfer Size Limits*, *Scheduled Transfer*, and *Transfer Data to Cloud Account* are not covered because the network assets in the language do not model bandwidth or traffic patterns. *Exfiltration Over Physical Medium* is not covered because we do not model situations where the attacker recovers removable media drives. Since most of these techniques revolve around evading detection when attempting to extract data, the conversation around bypassing defenses in the *Defense Evasion* Section 4.9 already explained how coreLang implements these mechanisms.

Two of the sub-techniques of *Exfiltration Over Alternative Protocol*, *Exfiltration Over Symmetric Encrypted Non-C2 Protocol* and *Exfiltration Over Asymmetric Encrypted Non-C2 Protocol* describe exfiltrating data over encrypted protocols. While they do not explicitly mention using encrypted protocols to avoid detection, the tactic itself does in its description. This behavior is implemented in the language by bypassing the `PayloadInspection` defense on `ConnectionRules` if `Data` that are encrypted could traverse them since we assume that the attacker would employ these two sub-techniques to avoid detection.

4.11. Tactics not covered

In the following, we elaborate on tactics that did not fit the previous categories but were neither incorporated into coreLang as they were not fitting.

The *Reconnaissance* tactic consists of techniques performed by adversaries to actively or passively gather information that can be used to support their imminent attacks.

Resource Development, on the other hand, consists of techniques that are performed by adversaries for creating, purchasing, or compromising/stealing resources that can be later used to support their attacks. Such resources include infrastructure, accounts, or capabilities. As already mentioned in Section 2.2, coreLang does not deal with an attacker's activity before an attack is performed[28] as the purpose of the language is to analyze the defender's system infrastructure.

Then, *Discovery* consists of techniques an adversary may use to gain knowledge about the system and internal network. The techniques in the *Discovery* tactic are often the precursors to other techniques that would target the identified assets. But as previously mentioned, coreLang assumes that the attacker is fully aware of the target environment, while a shortest path calculation is used for finding the attack path, and the assessment results are defender-oriented, meaning that they should be used to help the defender to identify weak points to improve the cyber resilience.

Persistence, *Privilege Escalation*, and *Defense Evasion* are conceptually tightly intertwined. To ensure long-term presence on a system, the attacker has to bypass existing defense mechanisms and/or modify the regular operations using elevated privileges to ensure that their tools are being run consistently. Again, due to the static nature of MAL languages attack graphs, *Persistence* is something we do not account for in this validation. However, it is feasible to conceptualize persistence as a form of impact. This perspective stems from the notion that once full access to a machine is secured, the ability to maintain that access, or persistence, could also be reasonably assured. Such an approach would integrate the concept of persistence into our analysis framework, linking it directly to the consequences of achieving complete system access.

Finally, the *Command and Control* tactic contains techniques that adversaries use to communicate with systems under their control within a victim network that is generally more specific than coreLang's representation. One example is the *Application Layer Protocol* technique that represents the communication of an attacker via any application layer protocol to avoid being detected by network filtering and intrusion detection systems. This decision represents a deliberate delimitation in our research focus. We concentrated primarily on assessing resilience against initial penetration, rather than examining the aspects of persistence and actions following an established penetration. This specific focus was chosen to address the crucial phase of initial defense, which is pivotal in preventing breaches and mitigating potential threats. This could also be too detailed for coreLang since, on the level of abstraction it has, it does not account for network protocols used for the communication.

5. Discussion

There are some potential objections to the work presented in this article. In this section, we will discuss what we consider to be the most important ones. Additionally, the aspects of validity, reliability, and generality in the context of this study will be addressed.

First, a possible objection could be about the inability of coreLang to model dynamic scenarios. MAL is static with respect to attackers' impact on the modeled architecture. Therefore, it is important to clarify that the inability to map the complete set of ATT&CK tactics into coreLang (as described in Section 4.11) can be attributed to the fact that the ATT&CK matrix's tactics are designed to encompass the dynamic nature of attack vectors and their prerequisites (as for example with the *Application Layer Protocol* technique under the *Command and Control* tactic), something that is beyond MAL's (which is the underlying framework for building coreLang), and therefore coreLang's modeling capabilities. This delimitation in coverage, is therefore inherent to the research project setup. On the same topic, it should be highlighted that this inherent static nature imposes limitations on dynamically incorporating assets into the attack graph. To mitigate some of these constraints, modelers can utilize speculative asset instances as a strategic workaround. As a reminder, speculative assets and defenses are not special categories, but rather existing assets and defenses added to the model based on hypothetical scenarios. Their purpose is to explore potential vulnerabilities and attack vectors that might arise under these speculative conditions. This approach facilitates the inclusion of potential changes within the model, contingent upon their prior identification as plausible by the modeler during the infrastructure modeling phase. These can then be represented as speculative instances, utilizing coreLang's speculative defenses (like the `notPresent` defense), thereby providing a means to proactively anticipate and prepare for potential emerging threats.

Furthermore, concerns about the precision of the attack graphs generated by coreLang could also be discussed. Our ambition with coreLang is to be able to predict potential attack vectors exploiting different techniques in coreLang instance models. In our mappings, we have clearly described how this would happen. We can therefore argue that coreLang enables the creation of both abstract/high-level, but also detailed models that are instrumentally correct. However, how precise predictions can be made from a specific model also depends on the resolution of the instance model created by the modeler. If, for example, an attack vector includes the compromise of a firmware, the firmware must also be explicitly introduced in the model. As demonstrated, this is possible given coreLang's capabilities so that an ambitious modeler can create a very precise model. Essentially we foresee a future where models can be constructed with high level of details by an expert on the respective component and later be reused by others a as a complex component. The purpose and goal of corelang was to provide good predictive capacity, which, as we argue is relatively good, but in practice, in such scenarios, it provides both greater explanatory and predictive capabilities.

Additionally, even though the completeness and correctness of coreLang against ATT&CK were probed in this work, a further completeness and correctness validation of the language could be performed to further support coreLang. In our most recent work (Katsikeas et al., 2024), that was conducted concurrently to this, we employed human domain experts whose experience in predicting cyber attacks was compared with assessments based on coreLang-generated attack graphs over a known insecure infrastructure. The key difference between those two works is that while this study is focused on coverage of the attack vectors, the other focuses on weighing/prioritizing the different individual attack vectors you can find in the generated attack graphs.

Regarding the validity of the mapping in regard to the introduction of the category level in the existing ATT&CK hierarchy, it is important to clarify that this was justified by providing valid reasoning for all the cases, and additionally, it does not change the semantics, it is just a syntactic placeholder introduced by us for convenience.

The reliability of our methodology for mapping coreLang against the ATT&CK matrix can be established by arguing and demonstrating that, for a given version of coreLang, the attack graphs generated for the same modeled infrastructure and attack goals (materialized as specific instance models) will consistently map to the relevant ATT&CK techniques.

In the next phase of our work, we aim to address certain concerns that have emerged as potential areas for future research. A primary concern is the preliminary nature of the probability distributions and their respective values assigned to the attack steps in coreLang. These distributions, as they currently stand, are based on prior assumptions made by us, rather than on extensive research. It is important to note that the ATT&CK framework also does not encompass details about the difficulty of achieving specific tactics, techniques, and procedures

[28] Furthermore, almost all of the techniques in the *Reconnaissance* and *Resource Development* tactics have only one mitigation associated with them, namely *Pre-compromise*, which explains that the (sub-)technique in question is hard to mitigate because "*it is based on behaviors performed outside of the scope of enterprise defenses and controls*" (MITRE, 2024).

(TTPs). Therefore, it is imperative that future research endeavors focus on establishing more precise and empirically validated values for these default probability distributions, both for the attack steps and the defenses in coreLang. This will enhance the accuracy and reliability of coreLang in representing real-world attack scenarios.

Finally, recognizing the limitations of coreLang, in the next paragraphs we will discuss and present our proposed ideas for further improving coreLang.

Introducing attacker reconnaissance capabilities in coreLang is a possible point of improvement, as this would be needed for attacker side simulations that capture a more realistic attacker's behavior. There are no inherent delimiting factors in MAL to achieve this. This can essentially be achieved if we add a new type of "search steps" in MAL, that will be associated with the time needed for the reconnaissance.

It is also pertinent to discuss certain limitations encountered in our mapping against the ATT&CK matrix, which could potentially be addressed in future iterations of coreLang. For instance, the introduction of a 'System Shutdown/Reboot' attack step, corresponding to the same-named technique in ATT&CK, is conceptually straightforward. However, by interpreting this as a system shutdown, we have consciously chosen not to incorporate this into the language, as it does not significantly enrich the generated attack graphs nor adds any additional information regarding the affected availability. If we, on the other hand consider the case of a reboot, encompassing it would require complex logic to handle attacker regression, as previously completed actions would need repetition after the system reverts to its initial state. This discussion underscores a broader principle: while some enhancements to the language are relatively simple to implement, others pose greater challenges or may even be infeasible. Thus, the development of coreLang involves careful consideration of the balance between feasibility and the value added by potential modifications.

Moreover, considering coreLang's intended role as a foundational language for future, more specialized derivatives, it is expected that these subsequent languages will not only cover some of the tactics not addressed in this current version but will also delve deeper into the sub-techniques pertinent to their specific domains. In line with this, our overarching design philosophy was to establish as generic a logical framework as possible, while simultaneously exploring avenues for further specialization. More specifically, as part of the methodology for developing coreLang, we sometimes did smaller experiments over how we would envision a more domain specific extensions as a means to verify that coreLang's architecture was sound. Throughout the many iterations of the development phase, our design decisions were predominantly guided by expert reasoning, although some were arrived at through consensus agreement.

6. Conclusion

As ICT infrastructures become increasingly complex, organizations are compelled to adopt robust methods such as threat modeling to assess their defenses and inform decision-making processes. In this work, we introduce the first release version of coreLang, a Domain-Specific Language (DSL) implemented in the Meta Attack Language (MAL), tailored for the ICT domain. coreLang is developed to support shortest path based attack graph analyses that are suitable for resilience/weakest link assessments of infrastructures and systems. This version of coreLang is validated against MITRE ATT&CK techniques, along with their associated impact and mitigations. This integration not only validates coreLang's capability to model and represent real-world attack scenarios, but also aids in pinpointing weaknesses. Furthermore, this paper provides a thorough description of the methodology utilized in this work, complete with an exhaustive discussion of the inherent assumptions and limitations.

From the validation results, we calculated that coreLang covers 46% (i.e., 104) of all the 224 techniques included in the ATT&CK (version 12.1[29]). If we exclude the tactics that are intrinsically not covered by coreLang (which are 63 in number), the coverage increases to 64%.

We contend that this advancement is of substantial utility, particularly for security engineers as it enables the automatic generation of ATT&CK-compliant attack graphs directly from system architecture models. This capability not only streamlines the process of identifying potential threats but also enhances the alignment of security measures with established frameworks.

CRediT authorship contribution statement

Sotirios Katsikeas: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology. **Andrei Buhaiu:** Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Conceptualization. **Mathias Ekstedt:** Supervision, Project administration, Methodology, Funding acquisition, Conceptualization. **Zeeshan Afzal:** Writing – review & editing, Writing – original draft, Methodology. **Simon Hacks:** Writing – original draft, Methodology, Conceptualization. **Preetam Mukherjee:** Writing – review & editing, Validation, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

A link to the models used for the mapping against MITRE ATT&Ck is provided in the article.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used OpenAI's language model ChatGPT in order to proofread and optimize parts of the text. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Acknowledgments

For a complete list of all the contributors of this project, the reader is referred to the GitHub repository of coreLang.[30]

This work was partly funded by the KTH Center for Cyber Defense and Information Security (CDIS), Sweden, the Swedish Civil Contingencies Agency (MSB), Sweden through the Swedish national research Center on Resilient Information and Control Systems (RICS), and the Swedish Energy Agency as part of the ERA-Net project HONOR, Sweden.

References

Åberg, Oscar, Sparf, Edvin, 2019. Validating the Meta Attack Language Using MITRE ATT&CK Matrix (Bachelor's thesis). URL http://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-259297.

Aksu, M. Ugur, Dilek, M. Hadi, Tatlı, E. İslam, Bicakci, Kemal, Dirik, H. İbrahim, Demirezen, M. Umut, Aykır, Tayfun, 2017. A quantitative CVSS-based cyber security risk assessment methodology for IT systems. In: 2017 International Carnahan Conference on Security Technology. ICCST, pp. 1–8.

Alam, Muhammad, Breu, Ruth, Hafner, Michael, 2007. Model-driven security engineering for trust management in SECTET. J. Softw. 2 (1), 47–59.

[29] https://attack.mitre.org/resources/updates/updates-october-2022.

[30] https://github.com/mal-lang/coreLang.

Basin, David, Clavel, Manuel, Egea, Marina, 2011. A decade of model-driven security. In: Proceedings of the 16th ACM Symposium on Access Control Models and Technologies. pp. 1–10.

Basin, David, Doser, Jürgen, Lodderstedt, Torsten, 2006. Model driven security: From UML models to access control infrastructures. ACM Trans. Softw. Eng. Methodol. (TOSEM) 15 (1), 39–91.

Buhaiu, Andrei, 2022. Corelang-documentation. https://mal-lang.org/coreLang-documentation/.

Buhaiu, Andrei, Katsikeas, Sotirios, Afzal, Zeeshan, 2022. Corelang-examples. https://github.com/mal-lang/coreLang-examples.

Dantu, R., Loper, K., Kolan, P., 2004. Risk management using behavior based attack graphs. In: International Conference on Information Technology: Coding and Computing, 2004. Proceedings. ITCC 2004. Vol. 1, IEEE, pp. 445–449, Vol.1.

Derbyshire, Richard, Green, Benjamin, Hutchison, David, 2021. "Talking a different language": Anticipating adversary attack cost for cyber risk assessment. Comput. Secur. 103, 102163.

Doynikova, Elena, Kotenko, Igor V., 2017. Enhancement of probabilistic attack graphs for accurate cyber security monitoring. In: IEEE SmartWorld, Ubiquitous Intelligence & Computing, Advanced & Trusted Computed, Scalable Computing & Communications, Cloud & Big Data Computing, Internet of People and Smart City Innovation, 2017. pp. 1–6.

FIRST, 2024. Common vulnerability scoring system v3.1: Specification document. https://www.first.org/cvss/v3.1/specification-document, (Accessed 02 January 2024).

Hacks, Simon, Butun, Ismail, Lagerström, Robert, Buhaiu, Andrei, Georgiadou, Anna, Michalitsi-Psarrou, Ariadni, 2021. Integrating security behavior into attack simulations. In: Reinhardt, Delphine, Müller, Tilo (Eds.), ARES 2021: The 16th International Conference on Availability, Reliability and Security, Vienna, Austria, August 17-20, 2021. ACM, pp. 120:1–120:13, URL https://doi.org/10.1145/3465481.3470475.

Hacks, Simon, Katsikeas, Sotirios, 2021. In: Rosa, Marcello La, Sadiq, Shazia W., Teniente, Ernest (Eds.), Towards an Ecosystem of Domain Specific Languages for Threat Modeling. In: Lecture Notes in Computer Science, vol. 12751, Springer, pp. 3–18.

Hafner, Michael, Breu, Ruth, Agreiter, Berthold, Nowak, Andrea, 2006. SECTET: an extensible framework for the realization of secure inter-organizational workflows. Internet Res. 16 (5), 491–506.

Hevner, Alan R., March, Salvatore T., Park, Jinsoo, Ram, Sudha, 2004. Design science in information systems research. MIS Q. 28 (1), 75–105.

Johnson, Pontus, Lagerström, Robert, Ekstedt, Mathias, 2018. A meta language for threat modeling and attack simulations. In: Proceedings of the 13th International Conference on Availability, Reliability and Security. In: ARES 2018, New York, NY, USA.

Jürjens, Jan, 2002. UMLsec: Extending UML for secure systems development. In: Jézéquel, Jean-Marc, Hußmann, Heinrich, Cook, Stephen (Eds.), UML 2002 - the Unified Modeling Language, 5th International Conference, Dresden, Germany, 2002, Proceedings. In: Lecture Notes in Computer Science, vol. 2460, Springer, pp. 412–425.

Jürjens, Jan, 2005. Secure Systems Development with UML. Springer Science & Business Media, Berlin, Heidelberg, Germany.

Katsikeas, Sotirios, Hacks, Simon, Johnson, Pontus, Ekstedt, Mathias, Lagerström, Robert, Jacobsson, Joar, Wällstedt, Max, Eliasson, Per, 2020. An attack simulation language for the IT domain. In: I.I.I., Harley Eades, Gadyatskaya, Olga (Eds.), Graphical Models for Security - 7th International Workshop, GraMSec 2020, Boston, MA, USA, June 22, 2020 Revised Selected Papers. In: Lecture Notes in Computer Science, vol. 12419, Springer, pp. 67–86.

Katsikeas, Sotirios, Johnsson, Pontus, Hacks, Simon, Lagerström, Robert, 2022. VehicleLang: A probabilistic modeling and simulation language for modern vehicle IT infrastructures. Comput. Secur. 117, 102705, URL https://www.sciencedirect.com/science/article/pii/S0167404822001031.

Katsikeas, Sotirios, Rencelj Ling, Engla, Johnsson, Pontus, Ekstedt, Mathias, 2024. Empirical evaluation of a threat modeling language as a cybersecurity assessment tool. Comput. Secur. 140, 103743, URL https://www.sciencedirect.com/science/article/pii/S0167404824000440.

Kordy, Barbara, Mauw, Sjouke, Radomirovic, Sasa, Schweitzer, Patrick, 2010. Foundations of attack-defense trees. In: Formal Aspects of Security and Trust - 7th International Workshop, FAST, Pisa, Italy, September 16-17, 2010. In: LNCS, vol. 6561, Springer, pp. 80–95.

Kordy, Barbara, Piètre-Cambacédès, Ludovic, Schweitzer, Patrick, 2014. DAG-based attack and defense modeling: Don't miss the forest for the attack trees. Comput. Sci. Rev. 13–14, 1–38.

Lin, Pengwen, Chen, Yonghong, 2018. Dynamic network security situation prediction based on Bayesian attack graph and big data. In: 2018 IEEE 4th Information Technology and Mechatronics Engineering Conference. ITOEC, IEEE, pp. 992–998.

Liu, Yu, Man, Hong, 2005. Network vulnerability assessment using Bayesian networks. In: Proc. SPIE. spie-5812, pp. 61–71.

Mauw, Sjouke, Oostdijk, Martijn, 2005. Foundations of attack trees. In: Information Security and Cryptology - ICISC, 8th International Conference, Seoul, Korea, December 1-2, 2005. In: Lecture Notes in Computer Science, vol. 3935, Springer, pp. 186–198.

2024. MITRE ATT&CK. https://attack.mitre.org/, (Accessed 02 January 2024).

MITRE, 2024. Pre-compromise. https://attack.mitre.org/mitigations/M1056/, (Accessed 02 January 2024).

MITRE, 2024a. CVE-2021-43890 detail. https://nvd.nist.gov/vuln/detail/CVE-2021-43890, (Accessed 02 January 2024).

MITRE, 2024b. Enterprise matrix. https://attack.mitre.org/matrices/enterprise/, (Accessed 02 January 2024).

Nordgren, Isak, Sederlin, Anton, 2021. Validating Enterpriselang: A Domain- Specific Language Derived from the Meta Attack Language Framework (Bachelor's thesis). URL https://kth.diva-portal.org/smash/record.jsf?pid=diva2:1596039.

Paja, Elda, Dalpiaz, Fabiano, Giorgini, Paolo, 2015. Modelling and reasoning about security requirements in socio-technical systems. Data Knowl. Eng. 98, 123–143.

Peffers, Ken, Tuunanen, Tuure, Rothenberger, Marcus A., Chatterjee, Samir, 2007. A design science research methodology for information systems research. J. Manage. Inf. Syst. 24 (3), 45–77, URL http://www.jstor.org/stable/40398896.

Phillips, Cynthia A., Swiler, Laura Painton, 1998. A graph-based system for network-vulnerability analysis. In: Proceedings of the 1998 Workshop on New Security Paradigms, Charlottsville, VA, USA, September 22-25, 1998. ACM, pp. 71–79.

Pirca, Ana Maria, Lallie, Harjinder Singh, 2023. An empirical evaluation of the effectiveness of attack graphs and MITRE ATT&CK matrices in aiding cyber attack perception amongst decision-makers. Comput. Secur. 130, 103254, URL https://www.sciencedirect.com/science/article/pii/S0167404823001645.

Poolsappasit, N., Dewri, R., Ray, I., 2012. Dynamic security risk management using Bayesian attack graphs. IEEE Trans. Dependable Secure Comput. 9 (1), 61–74.

Schneier, Bruce, 1999. Attack trees. Dr. Dobb's J. 24 (12), 21–29.

Shostack, Adam, 2014. Threat Modeling: Designing for Security. Wiley, Indianapolis, IN, USA.

Singhal, Anoop, Ou, Xinming, 2017. Security risk analysis of enterprise networks using probabilistic attack graphs. In: Network Security Metrics. Springer International Publishing, Cham, pp. 53–73.

Skobalj, Nedo, 2019. Validating Vehiclelang for Domain-Specific Threat Modelling of In-Vehicle Networks (Master's thesis). URL http://kth.diva-portal.org/smash/record.jsf?pid=diva2:1304913.

Strom, Blake E., Applebaum, Andy, Miller, Doug P., Nickels, Kathryn C., Pennington, Adam G., Thomas, Cody B., 2018. MITRE ATT&CK: Design and philosophy. URL https://www.mitre.org/sites/default/files/publications/pr-18-0944-11-mitre-attack-design-and-philosophy.pdf.

Tayouri, David, Baum, Nick, Shabtai, Asaf, Puzis, Rami, 2023. A survey of MulVAL extensions and their attack scenarios coverage. IEEE Access 11, 27974–27991.

Tuma, K., Calikli, G., Scandariato, R., 2018. Threat analysis of software systems: A systematic literature review. J. Syst. Softw. 144, 275–294, URL https://www.sciencedirect.com/science/article/pii/S0164121218301304.

van der Schoot, Willem, 2020. Validating Vehiclelang, A Domain-Specific Threat Modelling Language, from an Attacker and Industry Perspective (Master's thesis). URL http://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-284244.

Venable, John R., Pries-Heje, Jan, Baskerville, Richard, 2017. Choosing a design science research methodology. In: ACIS. University of Tasmania.

Xie, Peng, Li, Jason H, Ou, Xinming, Liu, Peng, Levy, Renato, 2010. Using Bayesian networks for cyber security analysis. In: 2010 IEEE/IFIP International Conference on Dependable Systems & Networks. DSN, pp. 211–220.

Xiong, Wenjun, Lagerström, Robert, 2019. Threat modeling - a systematic literature review. Comput. Secur. 84, 53–69, URL https://doi.org/10.1016/j.cose.2019.03.010.

Xiong, Wenjun, Legrand, Emeline, Åberg, Oscar, Lagerström, Robert, 2022. Cyber security threat modeling based on the MITRE enterprise ATT&CK matrix. Softw. Syst. Model. 21 (1), 157–177, URL https://doi.org/10.1007/s10270-021-00898-7.

Yimin, Cui, Junmei, Li, Wei, Zhao, Cheng, Luan, 2019. Research on network security quantitative model based on probabilistic attack graph. ITM Web Conf. 24, 02003.



Sotirios Katsikeas is a Ph.D. candidate at the School of Electrical and Computer Engineering at the Royal Institute of Technology (KTH) in Stockholm, Sweden. His main topic of research interests are domain specific languages for cyber attack modeling and simulations. He received his Diploma (5-year program) in Electrical and Computer Engineering from the Technical University of Crete in Greece and his MSc from the Royal Institute of Technology (KTH) in 2018. He is currently occupied as a software engineer for attack path simulations.



Andrei Buhaiu is a Research Engineer at Royal Institute of Technology (KTH) in Stockholm, Sweden. His main topic of research is cyber attack modeling and simulations. He received his M.Sc. in Computer and Information Engineering from Linköping University in 2013. He worked as a Software Engineer between 2012 and 2017.



Mathias Ekstedt is a Professor of Industrial Information and Control Systems at KTH Royal Institute of Technology in Stockholm, Sweden from where he also received his Ph.D. in 2004. His research interests include the intersection of cybersecurity and software systems architecture modeling and analysis. Mathias is the co-founder and director of KTH's Master programme in Cybersecurity.



Zeeshan Afzal is currently a Lecturer at the Department of Information Technology at Uppsala University, Sweden. Previously, he worked as a postdoctoral researcher in cyber security at the Royal Institute of Technology (KTH) in Stockholm, Sweden. His research interests include network security and threat modeling. Zeeshan received his Ph.D. (2014–2020) from Karlstad University, Sweden and his M.Sc. (2012–2014) from Chalmers University of Technology, Sweden.



Simon Hacks is associate professor for information systems at Department of Computer and Systems Sciences, Stockholm University, Sweden. His research interests lie in the quality of Enterprise Architecture (EA) and its models. Therefore, he has coined the term of EA Debt as an extension of Technical Debt, which provides a holistic view on organizations beyond technical aspects. Moreover, he performs research on the reuse of different models for attack simulations. In line, he facilitates the Meta Attack Language (MAL) that allows to provide domain specific languages (DSL) tailored to the needs of different stakeholders. Finally, he aims to provide tooling around MAL to ensure the quality of the created MAL DSLs. He received his Ph.D. from RWTH Aachen University, Germany in the field of EA modeling.



Preetam Mukherjee is working as an assistant professor at Digital University Kerala (DUK), India. Previously, he was a Postdoctoral Researcher with the Division of Network and Systems Engineering at KTH Royal Institute of Technology, Sweden. Preetam received his M.E. and Ph.D. in Computer Science from Jadavpur University, Kolkata, India, in 2011 and 2020, respectively. His research and development work is mainly focused on security modeling and analysis.